



Mitigating concept drift in data streams: an incremental decision tree approach

Hadi Tarazodar¹ · Karamollah Bagherifard¹ · Samad Nejatian² · Hamid Parvin³ · Razieh Malekhosseini¹

Accepted: 13 May 2024

© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2024

Abstract

While recognizing the significance of data in machine learning, we focus on addressing the challenge of concept drift, particularly in dynamic data streams. We propose an innovative incremental decision tree algorithm tailored for learning regression trees and model trees from evolving data streams. Vital to ensuring the quality and accuracy of predictive models is addressing this challenge. In this context, we present a novel solution: an incremental decision tree algorithm tailored for learning regression trees and model trees from time-varying data streams. Our algorithm is designed to operate at high speeds, effectively accommodating the influx of data at any scale, including scenarios with potentially unlimited data. Key innovations of our approach include a probabilistic defined sampling strategy that enhances the learning process and an advanced automatic method capable of handling non-stationary data distributions. However, the primary innovation lies in our methodology for detecting concept drift. Unlike conventional methods that reactively respond to increased errors, we introduce a proactive approach: monitoring the quality of individual subtrees by tracking their error evolution. This method allows us to detect changes in the objective function promptly, leading to timely adaptations in the model structure. Through extensive experimentation and evaluation, we demonstrate the effectiveness of our proposed algorithm in terms of prediction accuracy, model size, and change detection capabilities. Representing a significant advancement in the field of machine learning, particularly in addressing the challenge of concept drift in data streams, the proposed algorithm offers a competitive alternative to existing flow classifiers. Showcasing superior performance in terms of precision, recall, Fisher measure, and scalability, it underscores its potential to enhance decision-making processes across various domains by adapting swiftly to changing data patterns and maintaining high accuracy. The algorithm's innovative approach to incremental learning of decision rules, coupled with its adaptive extension for handling concept drift, holds promise for real-world applications where accurate and timely insights are paramount. Overall, the algorithm's robustness, adaptability, and efficiency position it as a valuable asset in stream data classification and decision support systems.

Keywords Machine learning · Concept drift · Data streams · Incremental decision tree · Regression trees · Hoeffding

✉ Karamollah Bagherifard
Ka.bagherifard@iau.ac.ir

Hadi Tarazodar
Hadi.tarazodar@gmail.com

Samad Nejatian
s.nejatian@gmail.com

Hamid Parvin
Parvin@iust.ac.ir

Razieh Malekhosseini
Malekhosseini@iauyasooj.ac.ir

¹ Department of Computer Engineering, Yasooj Branch, Islamic Azad University, Yasooj, Iran

² Department of Electrical Engineering, Yasooj Branch, Islamic Azad University, Yasooj, Iran

³ Department of Computer Engineering, Nourabad Mamasani Branch, Islamic Azad University, Nourabad Mamasani, Iran

1 Introduction

Machine learning, a burgeoning field in computer science, has experienced significant growth and found applications ranging from spam detection to robotics (Suárez-Cetrulo et al. 2022). At its core, machine learning relies on data to train algorithms, derive insights, and inform decision-making processes. The proliferation of the internet, social networks, and data-generating devices has led to an unprecedented influx of data (Abbasi et al. 2021), promising enhanced quality of life through data-driven decision-making. However, as data stream volume and velocity surge, so does the challenge of concept drift—a fundamental issue in learning from data streams. Concept drift arises from the dynamic nature of data streams, where underlying relationships and patterns evolve over time, posing a challenge for traditional machine learning models to maintain accuracy and relevance (Zenisek et al. 2019; Amin et al. 2023).

Various methods and algorithms have been introduced to address concept drift detection and model adaptation. Notably, algorithms based on the Hoeffding bound (Hoeffding 1963) have made strides in online learning settings (Wu et al. 2023). However, there remains a noticeable gap in addressing concept drift within tree-based regression models, including regression trees and model trees with deviance detection. Additionally, limited experimental work has explored online multi-objective models in the context of data streams.

To address these gaps and propose a novel approach to concept drift detection and adaptation in data streams, we present an innovative and incremental decision tree algorithm tailored for learning regression trees and model trees from dynamic data streams. Our approach introduces several pioneering elements distinguishing it from existing methods.

A cornerstone of our innovation lies in proactive concept drift detection. Instead of reacting passively to increased prediction errors, we propose a novel strategy to continuously monitor subtree quality and performance within the decision tree, enabling early detection of changes in the underlying objective function and prompting timely adaptations.

Our contributions extend beyond concept drift detection. We introduce a probabilistic defined sampling strategy to enhance the learning process, capturing relevant information more efficiently from data streams. Additionally, we present an advanced automatic method capable of handling non-stationary data distributions—a common occurrence in dynamic data streams.

Through comprehensive experimentation and evaluation, we showcase the superior performance of our

algorithm in terms of prediction accuracy, model size, and change detection capabilities. By addressing the pervasive challenge of concept drift, we offer a formidable competitor to existing flow classifiers and make a substantial contribution to the evolving field of machine learning (Wu et al. 2023; Weinberg and Last 2023).

The dynamic nature of data streams presents a formidable challenge in the realm of learning from such data, marked by the pervasive occurrence of concept drift and the pressing need for accurate change detection. Concept drift, characterized by the evolving patterns embedded within data streams, has emerged as a focal point of interest within the research community. Numerous methodologies for detecting concept drift have been introduced, ranging from standalone wrappers to integrated techniques within machine learning algorithms. Particularly, algorithms founded on the Hoeffding framework have been adeptly adapted for online non-resident learning settings, captivating the attention of a multitude of researchers. Nonetheless, a conspicuous void persists within the literature. To the best of our knowledge, prior endeavors have yet to explore the frontier of online methods meticulously crafted to cater explicitly to the nuanced domain of tree-based regression models. This void extends across the spectrum of tree-based regression, encompassing conventional regression trees, innovative model trees endowed with deviance detection capabilities, and decision trees tailor-made for regression tasks. Moreover, the absence of research in the realm of online consensus methods for regression within tree structures remains palpable. Additionally, the exploration of online multi-objective models remains largely uncharted territory, with empirical evaluations of their performance conspicuously absent from the research landscape (Wu et al. 2023).

This research serves as a beacon of innovation and makes multifaceted contributions to the field:

1. Foundational work in online regression trees: At its core, we lay the cornerstone for the development of online methods explicitly tailored for tree-based regression models. Our pioneering work seeks to address the paramount challenge of concept drift within this specific context, fundamentally reshaping the paradigm of online learning from data streams.
2. Innovative incremental algorithm: We usher in a new era with the introduction of an innovative incremental algorithm meticulously engineered to facilitate the learning of both regression trees and model trees. Remarkably, our algorithm extends its capabilities even to scenarios characterized by rapidly changing data streams. With a design crafted for handling high-velocity data streams, it possesses the potential to scale limitlessly.

3. **Advanced learning strategy:** Within our arsenal of innovations, we deploy a sophisticated probabilistic defined sampling strategy. This strategic deployment optimizes the learning process, allowing for the efficient extraction of pertinent information from the relentless flow of data streams.
4. **Adaptation to non-stationary data distributions:** Recognizing the intricate challenges posed by dynamic data streams, we present an advanced automatic method precisely tailored to tackle the complexities of learning within non-stationary data distributions. This adaptability proves indispensable in the face of ever-shifting data dynamics.
5. **Enhanced prediction accuracy:** Our overarching objective revolves around elevating the prediction accuracy of tree-based models to unprecedented levels. To realize this, we introduce a proactive approach to the detection of concept drift. In stark contrast to conventional methods that reactively respond to escalating errors, our method pioneers proactive monitoring of individual subtrees, continuously tracking their error evolution. This proactive stance empowers us to swiftly detect shifts in the underlying objective function, facilitating timely adaptations to the evolving data patterns.

The choice to adopt the Hoeffding tree algorithm over conventional decision trees stems from its ability to make early decisions with a notable degree of confidence while requiring lower computational overhead (Gama and Castillo 2004). Unlike traditional decision trees, which often necessitate processing large batches of data before making splits, the Hoeffding tree algorithm employs the Hoeffding bound to dynamically determine the point at which it can confidently make decisions based on a statistically significant subset of data. This adaptive nature allows the algorithm to make decisions incrementally as data streams in, without the need for exhaustive computations on entire datasets. By leveraging the Hoeffding bound, the algorithm achieves a balance between computational efficiency and decision accuracy, making it well-suited for handling concept drift in data streams where changes in data distribution occur over time. Therefore, the adoption of the Hoeffding tree algorithm aligns with the objective of our work to develop a methodology capable of detecting and adapting to concept drift in data streams efficiently and effectively.

This research endeavors to explore novel avenues in online learning for tree-based regression models, aiming to redefine the boundaries of machine learning within dynamic data stream contexts and revolutionize concept drift (Kelly et al. 1999) detection and adaptation processes.

Within a structured framework, our research systematically addresses the critical challenge of concept drift in data streams. The initial section provides a meticulous examination of existing literature, offering essential context for our contributions. Subsequently, our focus shifts to the elucidation of our proposed methodology, a comprehensive exploration designed to provide readers with a thorough understanding of its innovative features. Rigorous presentation of our findings follows in the experimental results section, where empirical evidence substantiates the effectiveness of our approach. Finally, the conclusion succinctly summarizes our contributions, emphasizing the transformative potential of our methodology in the realm of machine learning, especially concerning dynamic data streams and concept drift challenges.

2 Background and related work

The concept of adaptability in the realm of streaming data is of paramount significance, especially when dealing with the intricate phenomenon known as concept drift. Concept drift occurs when the fundamental statistical characteristics of a data stream undergo temporal shifts, rendering conventional statistical inference methods ineffective. Instead of assuming an immutable distribution within the target population, as often posited in statistical inference, concept drift acknowledges that these distributions are subject to flux and seeks to elucidate the repercussions of such fluctuations upon statistical inference (Tsymbal 2004).

One salient facet of concept drift is what we term population drift, signifying scenarios where the composition of a specific subpopulation within the dataset undergoes metamorphosis over time. This metamorphosis may manifest as alterations in class priors $P(c)$, class distribution $P(X|c)$, or the posterior distribution of class labels $P(c|X)$, where c represents the class, and X encapsulates the collective distribution of predictive features. These shifts wield the potential to exert profound impacts upon the efficacy of machine learning models and the field of statistical inference itself (Kelly et al. 1999; Gama and Castillo 2004).

Another variant of drift pertains to the cyclical resurgence of certain contextual frameworks, referred to as recurring context. Instances of recurring context are characterized by the periodic reappearance of specific concepts, akin to the rhythmic cadence of seasonal patterns in data (Zliobaite 2010). Distinguished within the literature, gradual drift emerges when a novel data source is gradually introduced in tandem with an incumbent source (Mimran 2013). As temporal epochs unfurl, the probability associated with the nascent data source undergoes incremental augmentation, precipitating a transformative shift within

the overarching data distribution. In parallel, incremental drift emerges as a distinctive variant of drift, characterized by the gradual and protracted evolution of data dynamics over an extended temporal trajectory (Cohen et al. 2008).

To confront these intricate facets of drift, change detection emerges as a ubiquitous and indispensable approach. It entails the vigilant monitoring of incoming data streams, attuned to subtle indicators of flux. Upon detection of a shift, the adaptive recalibration of models ensues, guided by the principle of change detection. Various techniques facilitate the identification of these shifts, encompassing the vigilant tracking of error rates and their juxtaposition with predefined threshold levels. An error rate breaching a stipulated threshold serves as an unequivocal signal of perturbed data distributions, beckoning the model to adapt accordingly (Pasquadibisceglie et al. 2023).

In summation, adaptability in the context of streaming data transpires as a nuanced endeavor, necessitating the astute discernment and mitigation of concept drift in its multifarious manifestations. Whether it takes the form of population drift, recurring context, or the gradual and incremental evolution of data dynamics, adeptly navigating these turbulent waters hinges upon the vigilant vigilance of change detection mechanisms. These mechanisms, underpinning the adaptability paradigm, furnish the means to calibrate machine learning models and ensure their resonance with the ever-shifting landscape of data distributions.

In the domain of managing concept drift, the technique of change detection assumes a prominent role, facilitating the graceful adaptation of machine learning models to the evolving data landscape. Its significance lies in its ability to discern substantial shifts within the data stream, thereby aiding in the model's adjustment process. These shifts typically manifest as an escalation in error rates, breaching predefined thresholds. Once identified, these alterations trigger the model's adaptive mechanism.

The scholarly literature has yielded two principal strategies for addressing such transitions (Cohen et al. 2008):

1. **Reinitializing the training data:** Certain methodologies, as demonstrated in Cohen et al. (2008), rely on statistical metrics as change indicators. When these metrics detect a surge in error rates, the algorithm embarks on retraining using the latest data. Another approach, outlined in Klinkenberg (2004), harnesses the Support Vector Machine (SVM) model. It entails resizing the sliding window's length and assigning weights to the data based on estimated error fluctuations.
2. **Employing a shadow model:** The concept of a shadow model involves the creation of an auxiliary sub-model

that operates alongside the primary model. Upon detecting a concept drift, a new model is instated. The shadow model assumes the mantle of the primary model should it exhibit superiority, notably in terms of enhanced accuracy. A sub-model may take the form of a sub-tree, trained in parallel with the primary decision tree. Should the shadow model surpass the performance of the primary model, the latter is superseded (Ikonovska et al. 2009; Bifet and Gavalda 2009).

2.1 Concept drift

Concept drift detection algorithms can be categorized based on their responsiveness into two main groups: sudden response and gradual response (Xu et al. 2017). These approaches often complement each other, with gradual response algorithms offering higher accuracy levels while sudden response algorithms sacrifice some accuracy (Xu et al. 2017). This trade-off typically arises from the inherent complexity of the algorithm, where achieving higher accuracy requires more time, reflecting the “no free lunch syndrome” principle.

In the realm of data stream concept drift, various approaches have been explored in the literature, including window-based, weight-based, and ensemble classifier methods (Xu et al. 2017). Some novel strategies adapt neural network structures to handle concept drift, such as Deep Evolving Fuzzy Neural Networks (DEVFNN) (Pratama et al. 2019a, b), Network with Dynamically Evolved Capacity (NADINE) (Pratama et al. 2019a, b), and MULTilayer Self-Evolving Recurrent Neural Network (MUSE-RNN) (Das et al. 2019).

Unlike these neural-based approaches, the proposed solution is specifically tailored for decision tree learning. Additionally, there are methods designed to address computational complexity, like Weakly Supervised Scalable Teacher Forcing Network (WeScatterNet) (Pratama et al. 2014). WeScatterNet effectively manages both sparse labeled samples and large-scale data streams using the distributed computing capabilities of the Apache Spark platform. However, it achieves this through label enrichment of partially labeled data samples, whereas our method relies exclusively on labeled data.

2.2 The Hoeffding tree algorithm

The Hoeffding tree algorithm is a machine learning approach tailored for the efficient construction of decision trees in the context of streaming data (Kumar et al. 2015; Banar et al. 2023; Svoboda et al. 2023). Unlike conventional decision tree algorithms that necessitate multiple passes through the entire dataset, the Hoeffding tree

algorithm processes data instances incrementally, one at a time, gradually assembling the decision tree structure. It leverages a statistical metric known as the Hoeffding bound to ascertain the minimum quantity of instances required for making a dependable decision when partitioning a node within the tree (Kumar et al. 2015). This strategy ensures the algorithm's swift adaptability to alterations in the data distribution, rendering it particularly suitable for scenarios characterized by continuous and high-volume data arrivals (Svoboda et al. 2023).

The primary advantage of the Hoeffding tree algorithm lies in its capacity to make early decisions with a notable degree of confidence, diminishing the necessity for exhaustive data scrutiny. This efficiency proves especially valuable in handling data streams, where traditional batch learning techniques become impractical due to the sheer magnitude and velocity of incoming data. Commonly employed in classification tasks, the Hoeffding tree algorithm finds utility in domains demanding real-time or near-real-time processing, including but not limited to network intrusion detection, financial transaction monitoring, and environmental surveillance (Kumar et al. 2015).

On the other end of the spectrum concerning the management of streaming data, we encounter the incremental family of algorithms, with the Hoeffding tree standing as a prominent representative. The original Hoeffding tree algorithm extends the principles of decision tree methodology, a widely favored nonparametric classification approach renowned for its swift evaluation speed and its adeptness in managing attributes of diverse types (Kumar et al. 2015). Furthermore, compact decision trees offer decision surfaces that are readily interpretable. The Hoeffding tree algorithm relies on the utilization of the Hoeffding bound, a statistical criterion employed to determine the minimum volume of training instances necessary to achieve a predetermined level of confidence.

Nevertheless, an inherent constraint of the Hoeffding tree algorithm becomes apparent in its assumption that the data distribution remains unchanged over time, a scenario seldom encountered in practical applications. To tackle this challenge, the same researchers introduced an enhanced algorithm named the Concept-Adapting Very Fast Decision Tree (CVFDT). The CVFDT incorporates a fixed-size window to identify nodes within the decision tree structure that are aging and might require updates, thus enabling the algorithm to adapt to shifting data distributions (Goncalves et al. 2014).

Streaming data poses challenges due to its sequential generation and high volume, limiting traditional ETL processes. Real-time analysis becomes imperative, and incremental model construction is necessary. In the context of Big Data, where continuous data streams are prevalent, swift concept drift responses are vital.

Two distinct algorithms, J48 and HAT, find applications in machine learning and data mining. J48, a decision tree algorithm, serves for classification and regression tasks, offering attributes such as interpretability, attribute selection, pruning, versatility, and ensemble capabilities. In contrast, HAT (Hoeffding Adaptive Tree) specializes in incremental learning within data streams, adapting to concept drift, ensuring memory efficiency, scalability, and real-time processing through the Hoeffding bound. While J48 suits batch data, HAT excels in dynamic, evolving data environments, with the choice contingent on specific data and learning requirements (Goncalves et al. 2014).

Goncalves et al. have introduced the EnHAT framework, which combines the J48 and HAT algorithms. In this particular context, J48, implemented in open-source Java and rooted in the C4.5 decision tree, is gaining considerable recognition for its skill in data classification (Salzberg 1993). In contrast, HAT embodies an adaptive tree algorithm that not only incorporates the principles of the Hoeffding tree method but also integrates the ADaptive WIndowing (ADWIN) algorithm to facilitate the detection of changes in data streams (Goncalves et al. 2014). The Hoeffding tree algorithm plays a central role in optimizing decision tree construction by strategically employing the Hoeffding bound to determine the required size of training samples necessary to achieve predefined confidence thresholds (Kumar et al. 2015). Noteworthy is ADWIN's additional contribution to the improvement of HAT, providing it with performance guarantees for effective error rate management (Goncalves et al. 2014).

The ensemble technique involves training multiple models on various datasets and making predictions using either the single best-performing model or a combination of the top-performing models. It's worth noting that each model within the ensemble can be created using a different algorithm. For instance, one ensemble approach utilizes an SVM model, as mentioned in Ouyang et al. (2009). Another method, known as WEAP-I in Ouyang et al. (2009), combines two ensemble techniques: Weighted Ensemble (WE), which generates individual classification models from different data chunks and utilizes model weights to create a single prediction model, and Averaging Probability (AP), which trains multiple classifiers on the most recent data chunk and equally aggregates them into a unified model.

Utilizing the Exponentially Weighted Moving Average (EWMA) (Lucas and Saccucci 1990) and adopting ADWIN (Goncalves et al. 2014) as drift detectors are among the methods for introducing a forgetting mechanism to the Hoeffding Tree. ADWIN stands out by providing performance guarantees concerning the obtained error rate, and both approaches demonstrate improved accuracy and reduced memory consumption compared to CVFDT.

However, it's essential to acknowledge that the extensions involving EWMA and ADWIN come at the cost of increased computational overhead in terms of the average processing time for each incoming example. This presents a trade-off between computational efficiency and memory conservation, a significant consideration in the realm of streaming data analysis.

Hoeffding-based regression tree and model tree algorithms have demonstrated success in addressing the challenge of learning regression models from continuous data streams. Building upon the principles of Hoeffding-based methods originally developed for classification, these algorithms leverage probabilistic estimates based on the Hoeffding bound. Several noteworthy algorithms in this category include FIMT, FIRT-DD, and FIMT-DD, as introduced in Ikonovska et al. (2008, 2009). FIMT is an online algorithm specifically designed for constructing linear model trees from constant data streams, ensuring the statistical stability of partition selection decisions through the utilization of Chernov bounds. An extended version, FIMT-DD, enhances FIMT by incorporating change detection capabilities to adapt to time-varying data streams and effectively identify temporal shifts. FIMT-DD, which builds upon the Hoeffding bound, serves as a crucial reference for presenting the core concepts in this context. Notably, the inductive inference process in FIMT-DD closely resembles the inference procedures found in decision tree learning algorithms rooted in Hoeffding, with two significant distinctions. The first key distinction lies in the approach to acquiring probability estimates through the comparison of candidate distributions, while the second key difference involves the incremental inference of additional linear models within the tree's leaves (Ikonovska et al. 2008, 2009).

2.3 Processing numerical attributes

Processing numerical attributes efficiently is crucial in machine learning algorithms, especially concerning the selection of split points. The effectiveness of the split selection method is significantly influenced by the number of split points. In the case of numerical attributes with a high number of distinct values, both memory consumption and computational costs rise substantially. In batch settings, a common approach involves a preprocessing stage, which entails categorizing a broad spectrum of numerical (discrete) attributes. However, this method necessitates an initial data verification step before learning, as well as resource-intensive sorting operations. Implementing a separate preprocessing step before learning becomes challenging in a streaming context, where operational sorting becomes prohibitively expensive. Moreover, the wide range of potential values for numerical attributes is often

unknown and subject to variation with shifts in data distribution.

In addressing classification tasks within data streams, various innovative solutions have been proposed, each offering unique approaches to cope with the challenges posed by continuous and dynamic streaming data. One method involves online discretization with a predefined number of bins (Domingos and Hulten 2000). This technique dynamically determines sub-range boundaries based on the first n unique values observed in the data sequence. As new unique values emerge, new bins are created until reaching a predetermined limit of n seeds. However, it's important to note that this method, relying on the data sequence prefix for boundary determination, may be sensitive to unbalanced data distributions.

Another category of methods focuses on Gaussian-based approaches for selecting continuous partition tests in both two-class (Gama and Castillo 2004) and multi-class problems (Pfahring et al. 2008). One specific method, presented in Gama and Castillo (2004), utilizes a modified version of quadratic discriminant analysis. This approach assumes that the distribution of attribute values conforms to a normal distribution for both classes.

Additionally, a comprehensive approach based on binary search trees has been proposed (Gama et al. 2014). Binary search trees provide an efficient organizational structure for data search and are particularly well-suited for handling continuous data streams.

These innovative methods offer diverse strategies for addressing challenges such as unbalanced data distributions, computational efficiency, and minimal storage requirements, contributing to the effective adaptation of classification models to the dynamic nature of data streams. The central idea is expressed in Eq. (1):

$$p_1 \emptyset\{\mu_1, \sigma_1\} = p_2 \emptyset\{\mu_2, \sigma_2\} \quad (1)$$

where $\emptyset$ represents the probability density function of the normal distribution, and p_1 or p_2 denotes the estimated probability that a sample belongs to class c_1 or c_2 . This method proceeds by dividing the numerical features into three subranges: $(-\infty, d_1)$, (d_1, d_2) , and $(d_2, +\infty)$, where d_1 and d_2 are potential roots of the equation. The root that is equidistant from the means of both classes is selected as the division point, which is numerically evaluated for specific attributes. Consequently, this method relies solely on the mean and variance values for each class of each numerical attribute. These statistics are estimated from a sample of examples. The multi-class variant, as presented in Weinberg and Last (2023), partitions the range between the minimum and maximum observed values into n segments (e.g., $n = 100$). For each potential division, the method estimates the parameters of the Gaussian distribution for each class, subsequently employed to estimate the

distribution on both sides of the division (based on the percentile under the normal distribution in each class). Notably, methods based on this Gaussian approximation are computationally efficient compared to the comprehensive method outlined in Gama and Castillo (2004).

This approach further involves a binary tree structure created for each numerical attribute. Each node, corresponding to a potential partition structure, encompasses a wide range of attributes. Additionally, each node maintains two arrays containing the number of instances for each computation class from the beginning of the sequence. The first array tracks samples that have traversed the node with values less than or equal to the division value, while the second array is associated with samples having values greater than the division point. When a previously unseen value k enters the system, it is placed in the right subtree below the node with the smallest split value greater than k . Consequently, the size of the tree is influenced by the number of unique values n . The insertion of new values into the binary tree structure has an average time complexity of ($O(\log n)$) and a worst-case time complexity of ($O(n)$).

Building upon this method, the Extended Binary Search Tree (EBST) structure has been developed, specifically tailored for regression trees. The core concept of this approach involves maintaining an E-BST for each numerical feature within the leaves of the regression tree. E-BST preserves essential statistics and facilitates value sorting. Each E-BST node includes a test related to the feature range (in the form of $\leq$ key), with keys representing unique values from the corresponding numeric attribute domain. Additionally, each E-BST building node stores two arrays, each comprising three elements. The first array retains statistics for samples that reach the node with attribute values less than or equal to the key, encompassing (1) the count of samples that have reached the node, (2) the sum of the values of the target entity $\sum y$, and (3) the total values of the squared objective feature $\sum y^2$. The second array preserves these statistics for instances reaching the node with attribute values greater than the key. E-BST structures are updated with the arrival of each new instance accessing the leaf of the regression tree where they are located. This process commences with an empty E-BST, and unseen attribute values are inserted into a structure resembling a binary search tree. A noteworthy distinction in E-BST traversal is that the count is updated at all nodes.

Incremental linear model trees, also known as incremental regression trees, are a type of machine learning model designed for handling continuous and evolving data streams. These trees are particularly well-suited for regression tasks, where the goal is to predict a continuous target variable based on input features.

In the previous sections, we discussed various class methods for learning linear model trees, categorizing them based on their approach to solving the selection decision. These methods take into account the nature of predictions being calculated using linear models rather than fixed regressors and utilize different least squares evaluation functions. It's worth noting that in this context, there is no assurance that the chosen partition will minimize the mean squared error of the model tree.

One set of algorithms aims to minimize the least squares error by considering hypothetical linear models on both sides of the candidate division. However, another set of algorithms seeks to provide theoretical guarantees for minimizing the mean squared error of the model tree. Nevertheless, these algorithms come with significantly higher computational complexity compared to the first group. Furthermore, attempting to fit linear models on both sides of the candidate division can be intractable for real (numerical) traits.

Given these considerations, we have adopted an approach that remains agnostic regarding the presence of linear models in the leaves of the tree. Consequently, our model tree construction resembles that of a regression tree, employing standard deviation reduction as the evaluation function. Linear models are computed subsequent to segmentation using a computationally lightweight method based on online learning of unthresholded perceptrons. This approach strikes a balance between accuracy and computational efficiency, making it well-suited for practical applications where both factors are crucial.

Here are some key characteristics and concepts associated with incremental linear model trees:

1. Continuous learning: Incremental linear model trees are capable of continuously learning and updating their model with new incoming data points. Unlike traditional batch learning methods that require retraining the entire model when new data arrives, incremental trees incrementally adapt to changes in the data distribution.
2. Tree structure: Similar to decision trees, incremental linear model trees have a hierarchical tree-like structure. Each node in the tree represents a linear model or a regression function that estimates the target variable based on a subset of features.
3. Splitting criteria: Nodes in the tree are split based on specific criteria, which can vary depending on the algorithm and the problem at hand. The splits aim to partition the feature space into regions where linear models can make accurate predictions.
4. Linear models: Linear regression models are often used at the leaf nodes of the tree. These models capture the

linear relationship between the input features and the target variable within each region of the feature space.

5. **Concept drift:** Incremental linear model trees are designed to adapt to concept drift, which refers to changes in the data distribution over time. When concept drift is detected, the tree structure can be updated to accommodate the new data patterns.
6. **Scalability:** These trees are designed to be scalable and memory-efficient, making them suitable for real-time and high-volume data stream applications.
7. **Prediction:** Predictions are made by traversing the tree from the root node to a leaf node that corresponds to the region of the feature space where the input data point falls. The linear model at the leaf node is then used to make the prediction.
8. **Applications:** Incremental linear model trees are applied in various domains, including financial forecasting, sensor data analysis, and any scenario where continuous and accurate regression on evolving data is required.

Overall, incremental linear model trees provide a flexible and adaptive approach to regression tasks in dynamic environments, making them valuable tools for handling data streams with changing data patterns. Different algorithms and variations exist for implementing these trees, each with its own advantages and trade-offs.

2.4 Meta-heuristic algorithms

Meta-heuristic algorithms play a crucial role in addressing complex optimization problems where traditional methods may be ineffective or impractical. In the context of the present study, meta-heuristic algorithms could be employed to optimize various aspects of the proposed incremental decision tree algorithm. For example, meta-heuristic algorithms such as genetic algorithms, particle swarm optimization, simulated annealing, or ant colony optimization could be utilized to optimize hyperparameters, feature selection, or rule generation processes within the decision tree algorithm. These meta-heuristic approaches offer the advantage of exploring a vast search space efficiently, enabling the algorithm to find high-quality solutions even in dynamic and evolving data stream environments. By integrating meta-heuristic algorithms into the proposed framework, researchers can further enhance the algorithm's performance, scalability, and adaptability, ultimately advancing the state-of-the-art in stream data classification and decision support systems.

2.5 Stat-of-the-art methods

In this section, we introduce some of the latest methods. To support the relevance of our proposed methodology for addressing concept drift in data streams and processing numerical attributes, we incorporate insights from Analysis of community question-answering issues via machine learning and deep learning: State-of-the-art review (Kumar et al. 2015). This comprehensive review explores the utilization of machine learning and deep learning techniques in analyzing content from community question-answering sites. By summarizing key themes in CQA research related to questions, answers, and users, the review provides valuable insights into the application of machine learning methodologies in handling diverse datasets and extracting meaningful information from dynamic content sources.

Another study examines the impact of the COVID-19 pandemic on the 2021 Hajj season and its influence on sentiment expressed on Twitter (Gutub et al. 2023). With restrictions limiting the number of pilgrims for safety, activities in Mecca and Medina were affected. Twitter became a key communication platform for discussing issues related to the pandemic during the Hajj season. Analyzing more than 22,000 tweets from Makkah and Madinah, this study utilized a CNN-LSTM deep learning model to classify tweets into positive, negative, and neutral sentiment. The results show varied negative sentiments towards the pandemic in tweets from these cities, with a strong correlation between the sentiments expressed in Makkah and Madinah, providing valuable insights into residents' perceptions of the pandemic.

To enhance the effectiveness of blind image steganalysis, this study proposes a method utilizing third-order SPAM features and an ensemble classifier (Hemalatha et al. 2023). The success of blind steganalysis relies on accurately extracting statistical footprints from image features and selecting appropriate machine learning (ML) algorithms. Despite the consideration of numerous features, many steganalysis approaches report suboptimal detection accuracy. This research addresses this issue through a three-step process. First, curvelet denoising is employed to suppress natural noise residuals and produce stego noise residuals. Next, Third-order Markov-chain sample transition probability matrices are extracted as features. Finally, an oblique decision tree ensemble with a multisurface proximal support vector machine (SVM) classifier is employed to achieve superior detection accuracy compared to state-of-the-art classifiers.

To explore the practicality of blind image steganalysis using feature-based classification, this study investigates the effectiveness of statistical features and artificial neural network techniques (Aljarf et al.). The research focuses on

detecting hidden data in gray images through the extraction of features such as correlation, contrast, homogeneity, and energy from the gray level co-occurrence matrix (GLCM). Experimental evaluations are conducted using stego-images generated via LSB steganography, with machine learning (ML), radial basis function (RBF), and naïve Bayes classifiers employed to categorize the observations and improve detection accuracy. The results demonstrate the reliability and enhanced detection rate of the proposed system across various steganographic methods, particularly with the correlation features exhibiting increased accuracy when combined with RBF and naïve Bayes classifiers.

To ensure the completeness of our research coverage and the applicability of our proposed approach to various urgent applications, we recommend incorporating the following references: “Velocity-Based Modeling of Physical Interactions in Dense Crowds” (Kim et al. 2014), “Redefining Food Safety Traceability System through Blockchain: Findings, Challenges, and Open Issues” (Gutub et al. 2023), “Automating Global Threat-Maps Generation via Advancements of News Sensors and AI” (Abu-Hashem and Gutub 2022), and “Efficient Computation of Hash Hirschberg Protein Alignment Utilizing Hyper Threading Multi-Core Sharing Technology” (Abu-Hashem and Gutub 2022). These references cover a wide range of topics including crowd modeling, food safety traceability, threat detection, and bioinformatics, offering valuable insights and methodologies that can complement our proposed methodology for addressing concept drift in data streams and processing numerical attributes, and expanding its applicability to diverse real-world scenarios.

3 Proposed method

The proposed method recognizes that learning from data streams constitutes an ongoing, infinite dynamic process, encapsulating a singular cycle comprising data collection, the learning process, and model validation. This approach fundamentally distinguishes itself from traditional supervised machine learning, wherein samples are readily available as static datasets. Instead, data streams continuously provide samples in an unending manner, potentially extending indefinitely. This poses several formidable challenges:

1. **Storage limitations:** Given the substantial volume of data generated, it is often impractical to store all incoming data before processing. This necessitates real-time processing and learning capabilities to extract valuable insights without the luxury of batch processing.

2. **Temporal dynamics:** Data collected over extended periods undergo temporal shifts, rendering relationships and patterns learned from earlier data potentially obsolete for the current model. For instance, trends in social networks are in constant flux due to various factors, making it essential to adapt to evolving data patterns.
3. **Noise and conflict handling:** Data stream algorithms must contend with noise and conflicting information as they arrive in real-time. Dealing with these challenges introduces an additional layer of complexity, particularly when striving to provide theoretical guarantees regarding the performance of online learning algorithms.

Addressing these challenges requires the development of innovative methods and algorithms that not only enable real-time processing of vast data streams but also incorporate mechanisms for concept drift detection, adaptation to changing data dynamics, and robust handling of noisy and conflicting information. The proposed method aims to tackle these challenges head-on, providing a comprehensive solution for learning from data streams with an emphasis on accuracy, adaptability, and efficiency.

The stream data mining community has approached the challenge of processing data streams from a practical standpoint, outlining a set of requirements that algorithms must fulfill to effectively handle these continuous data streams. Researchers in Breiman (2017) identify the key features that an ideal data flow algorithm should have:

- (1) **Incremental data processing:** The algorithm should be capable of processing data incrementally as it arrives.
- (2) **Gradual model update:** The model should gradually incorporate new information without necessitating the rebuilding of the entire model, enabling adaptability to evolving data patterns.
- (3) **High convergence speed:** The algorithm should exhibit high convergence speed, ensuring that the model evolves rapidly to reflect changes in the data distribution.
- (4) **Limited sample processing time:** The processing time for each incoming sample should be constrained to ensure real-time responsiveness.
- (5) **Constant resource consumption:** Ideally, the algorithm’s CPU and memory consumption should remain constant and independent of the number of processed samples, ensuring scalability and efficiency.
- (6) **Concept drift adaptation:** The algorithm must be capable of reacting and adapting to changes in the data distribution as they occur, particularly in cases

of concept drift, where the underlying data patterns change over time.

While feature 5 can be achieved through various learning schemes, including collective learning, which accumulates categories over time and creates new models as memory fills up, incremental methods are typically preferred in the data stream literature due to their superior results. Incremental decision trees are a well-established method for data stream processing and are central to more powerful methods such as classifier assembly. Additionally, the k -nearest neighbors (kNN) method, which finds the nearest neighbors based on distance criteria, is another popular option. However, the original kNN method stores samples internally, making it unsuitable for real-time stream settings. Various extensions to kNN have been proposed to address these limitations in handling potentially limitless data streams.

Feature 6 relates to the ability to change the functionality of the current model in response to concept drift. Concept drift occurs when old data becomes irrelevant or detrimental to the current model, degrading its performance. This phenomenon is especially prevalent in evolving data streams. To address this challenge, this section presents a method based on an assembly of incremental decision trees for data streams. This method encompasses five primary phases:

- (1) Low latency multithreaded consensus learning for dynamic data streams: This phase focuses on achieving low latency and efficient multithreaded consensus learning to handle dynamic data streams.
- (2) Leaves and counters: The second phase involves the management of leaves and counters within the decision trees.
- (3) Consensus of incremental regression trees: Here, the method addresses the consensus building process for incremental regression trees.
- (4) Multi-threaded collective learning: This phase involves multi-threaded collective learning, enhancing the model's adaptability and scalability.
- (5) Sample buffer: The final phase introduces a sample buffer to manage incoming data samples effectively.

By addressing these phases and features, the proposed method aims to provide a robust and efficient solution for handling concept drift and maintaining the quality of models in data streams, contributing to the advancement of real-time data mining and machine learning in dynamic environments.

Phase 1: low latency multithreaded consensus learning for dynamic data streams

In this section, we delve into the mathematical and algorithmic foundations of Phase 1, which is dedicated to

enhancing the Hoeffding Tree (HT) for efficient learning from dynamic data streams. HT is an incremental decision tree known for its capability to adapt swiftly and accurately to data streams. Rather than relying on ensemble methods, which often combine multiple single learners like standalone HT models, Phase 1 focuses on leveraging HT as a standalone model for data stream processing.

The core concept of HT is based on the idea that analyzing a small, randomly selected subset of data, denoted as S , is often sufficient for informed decision-making regarding model expansion. This concept hinges on the Hoeffding inequality, which provides a probabilistic guarantee that the observed sample mean of a variable is close to its true mean with a high probability. Specifically, for binary classification tasks, the Hoeffding bound can be expressed as:

$$P(|\bar{X} - \mu| \geq \varepsilon) \leq 2e^{-2\varepsilon^2 n}, \quad (2)$$

where, P is the probability, $\bar{X}$ is the sample mean of the binary class labels, μ is the true mean (0.5 for binary classification), ε is the Hoeffding bound (a small positive value), n is the number of samples in the subset S .

This inequality implies that, with high probability, the observed error rate of a node in the tree is close to its true error rate.

In this context, the primary goal is to determine the optimal subset size, S , which minimizes computational cost while ensuring accurate learning. This is often referred to as the “splitting criterion.” One common approach is to select S that minimizes the Hoeffding bound, ensuring that the chosen subset is large enough to make statistically significant decisions about the split. This criterion is mathematically expressed as:

$$|S| = \max\left(\frac{4R^2 \ln(2/\delta)}{\varepsilon^2}, 1\right), \quad (3)$$

where, R is the range of class probabilities (0.5 for binary classification), δ is the confidence parameter, which determines the probability of the true error rate falling within the Hoeffding bound. Thus, the algorithm aims to determine the minimum $|S|$ required to make a statistically confident decision while optimizing computational efficiency.

In contrast to relying solely on a single category (single HT model), this approach advocates employing an assembly of incremental HT classifications, collectively contributing to the final prediction. Collective learning, facilitated through an ensemble of models, is favored for data stream processing due to its superior predictive performance compared to individual models. In this article, the ensemble is formed through a combination of incremental regression trees.

The key objectives of this phase encompass both algorithmic and implementation enhancements to HT and its combinations. The aim is to boost throughput and reduce resource consumption while either maintaining or enhancing prediction accuracy. The overarching goal is to enhance efficiency by mitigating latency through the implementation of a multithreaded ensemble of regression trees.

The proposed design is characterized by modularity, scalability, and compatibility across a range of hardware and operating systems. This innovative approach harnesses the potential of optimized cache design, exploits SIMD (Single Instruction, Multiple Data) vector capabilities within functional units, and leverages multiple processor cores. While recent years have seen exploration into the parallelization of decision trees and classification sets, existing solutions fall short of meeting the rigorous demands of real-time data streams.

The core of the proposed binary tree data structure remains agnostic with respect to the implementation of leaves and counters. It is meticulously designed to ensure cache compatibility, compressing a primitive binary subtree of a specific depth into a single cache line of the L1 processor. When a node is requested by the processor, it retrieves a cache line from L1 containing an entire subtree. This strategic approach minimizes access to main memory and maximizes cache memory utilization. For instance, Fig. 2 illustrates the division of a binary tree into two subtrees, each stored in a distinct cache line. In this illustrative example, each subtree has a maximum height of 3, resulting in a maximum of 8 leaves and 7 internal nodes. Furthermore, leaves have the capacity to point to root nodes of other subtrees, enhancing the efficiency of memory management. The pseudocode for phase 1 of the proposed method is shown in Fig. 1.

Phase 2: leaves and counters

In this phase, we delve into the mathematical underpinnings and algebraic expressions governing the leaves and counters within our innovative approach to dynamic data stream learning. These components are instrumental in achieving efficient and precise model learning.

Let's introduce formal mathematical notations:

L : Total number of labels, where $0 \leq j < L$.

N : Total number of attributes, where $0 \leq i < N$.

V_0 : Total number of possible attribute values, where $0 \leq k < V_0$.

Each leaf node in the decision tree architecture corresponds to an instance of a data structure, denoted as D . This data structure encapsulates all the essential information required for calculating the performance of the partitioning criterion and executing leaf classification.

The central element in our proposed design is the concept of leaf counters, denoted as $Counter(j, i, k)$. Each

counter corresponds to a specific label j , attribute i , and attribute value k . Its purpose is to tally the occurrences of attribute i assuming the value k within label j . Formally, this can be expressed as:

$$Counter(j, i, k) = \sum (x_{ij} = k) \quad (3)$$

Here, $\sum$ represents the summation operator, and x_{ij} denotes the attribute i for label j within the dataset. The expression $\sum (x_{ij} = k)$ computes the total count of occurrences where attribute i assumes the value k within label j .

Additionally, we introduce the concept of a total attribute count, denoted as $Total(j, i)$. This calculates the overall count of attributes observed for a given label j . This is essential for determining how many times each attribute i has the value 0 within that label. Mathematically, we represent this as:

$$Total(j, i) = \sum (x_{ij}) \quad (4)$$

In this expression, $\sum (x_{ij})$ computes the total count of occurrences of attribute i within label j .

Our memory organization strategy for attribute counters involves sequential storage for each label, with each counter occupying a specific number of bits (typically 32 bits in our implementation). This design choice enables us to leverage SIMD (Single Instruction, Multiple Data) registers and instructions available in modern processors, such as Intel AVX2.

The SIMD optimization technique allows parallel operations on these counters. For example, within SIMD registers, we can fit up to 8 counters per register, and operations can be executed concurrently. This vertical parallelism occurs between two SIMD registers, enabling efficient computation across different labels, attributes, and attribute values.

Furthermore, horizontal parallelism takes place within a single SIMD register, facilitating operations involving different attributes or values within the same label. The arithmetic operations primarily involve addition, multiplication, and division, all of which exploit the SIMD instruction set for efficient computation.

In summary, Phase 2 revolves around the leaves and counters of our model, which are pivotal for achieving computational efficiency and precision in learning from dynamic data streams. The mathematical formulations and algebraic expressions provided here lay the groundwork for efficient computation, ensuring that our approach adeptly addresses the challenges posed by real-time data stream processing. The pseudocode for phase 2 of the proposed method is shown in Fig. 2.

Phase 3: consensus of incremental regression trees

In this phase, we address the critical task of achieving a consensus among incremental regression trees to form an

Fig. 1 Pseudocode for phase 1 of the proposed method

Algorithm 1: Phase 1 - Low Latency Multithreaded Consensus Learning

Input:
 DataStream: Input data stream
 Hoeffding Trees: List of incremental Hoeffding Trees
 Number of Threads: N

Output:
 Final Prediction: Consensus prediction

Initialize N worker threads

For each worker thread i from 1 to N do:
 Create a sample buffer $SampleBuffer_i$
End For

while DataStream is not empty do:
 For each worker thread i from 1 to N do:
 Read next data sample $Data_i$ from DataStream
 Add $Data_i$ to $SampleBuffer_i$
 End For
 End while

For each Hoeffding Tree $learner_j$ in Hoeffding Trees do:
 Train $learner_j$ asynchronously on $SampleBuffer_j$
End For

Wait for all threads to finish training

Initialize Predictions array $Predictions_j$ for each $learner_j$

For each $learner_j$ in Hoeffding Trees do:
 $Predictions_j = Predict(learner_j, Data_i)$ for each $Data_i$ in $SampleBuffer_j$
End For

$FinalPrediction = CombinePredictions(Predictions_1, Predictions_2, ..., Predictions_L)$

Output $FinalPrediction$

End Algorithm

accurate and interpretable classifier. Our objective is to harness the power of multiple trees and merge their outcomes into a cohesive, robust prediction model.

The essence of this phase lies in developing an algorithm that effectively combines the outputs of individual incremental regression trees. As illustrated in Fig. 3, this consensus-building process involves a collective decision-making mechanism. A total of n incremental regression trees collaborate to produce results, which are subsequently aggregated using an agreement function. Ultimately, the consensus output is determined through a majority voting scheme.

The process of classification unfolds as we traverse the decision tree structure, starting from the root and progressing towards the leaf nodes. At each intermediate node, a division decision is made, guiding us either to the left or right branch. This decision-making process is governed by the principles of incremental regression tree mechanisms, detailed in previous sections.

To evaluate the quality of classification decisions at each node, we introduce a loss function, formally defined as follows:

Let $D = X, y$, where $X = [x_1, x_2, \dots, x_n]$ and $y = [y_1, y_2, \dots, y_n]$. Here, x_i represents the feature vector, and y_i signifies the label at the current node, denoted as m . The reduction in error $\Delta(y_i)$ is computed by subtracting the loss function's evaluation after division from the loss function's evaluation before division. This can be expressed as:

$$\Delta(y_i) = \mathcal{L}(y_i) - \left[\frac{n_l}{n} \mathcal{L}(y_l) + \frac{n_r}{n} \mathcal{L}(y_r) \right] \quad (5)$$

where, $\Delta(y_i)$ denotes the reduction in error upon splitting at node m , $\mathcal{L}(\cdot)$ signifies the loss function, commonly employed in classification problems, such as the Gini loss function or entropy loss function, n_l and n_r represent the numbers of samples directed to the left and right branches, respectively.

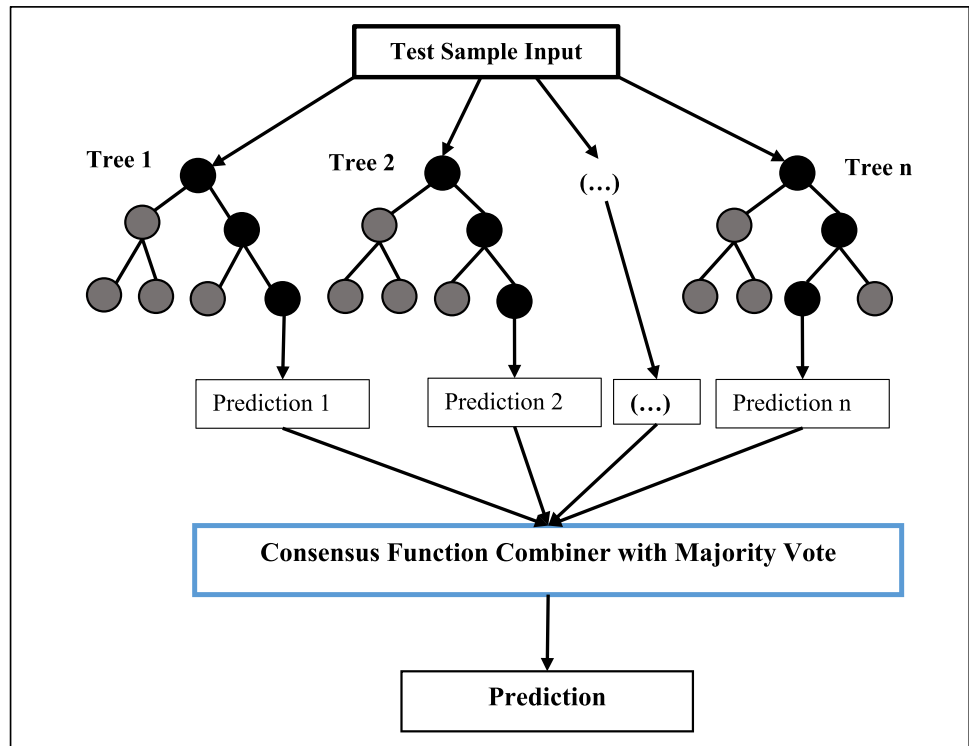
In addition to elucidating the consensus-building process, this phase introduces the concept of achieving diversity within the ensemble of trees. Rather than relying

Fig. 2 Pseudocode for phase 2 of the proposed method**Algorithm2: Phase 2 - Leaves and Counters****Input:** L : Total number of labels N : Total number of attributes V_0 : Total number of possible attribute values**Output:**

Leaf Counters: Counters for each label, attribute, and attribute value

Total Attribute Counts: Total counts of attributes for each label

Initialize Leaf Counters and Total Attribute Counts arrays

For each label j from 0 to $L-1$ **do**: **For** each attribute i from 0 to $N-1$ **do**: **For** each attribute value k from 0 to V_0-1 **do**: $LeafCounters[j][i][k] = 0$ // Initialize counters to zero**For** each label j from 0 to $L-1$ **do**: **For** each attribute i from 0 to $N-1$ **do**: **For** each data sample D in the dataset **do**: **If** $D.label = j$ **then**: $TotalAttributeCounts[j][i] += 1$ **For** each attribute value k from 0 to V_0-1 **do**: **If** $D.attributes[i] = k$ **then**: $LeafCounters[j][i][k] += 1$ **End Algorithm****Fig. 3** Consensus mechanism of regression decision trees in the proposed method

on diverse subsets of samples generated by sampling methods, our approach focuses on creating diversity within the ensemble itself. This diversity encompasses members comprising distinct regression trees, dynamically and

adaptively generated in real-time based on the evolving properties of the incremental regression trees.

The diversity of classifiers within the ensemble ensures that each classifier offers unique predictions. Typically,

combining such diverse classifiers results in enhanced overall performance. A common method for aggregating predictions from ensemble members is through a majority vote. In its simplest form, this involves a democratic system where the final decision is based on the majority's consensus.

However, our approach goes further by allowing for the assignment of different weights to individual classifiers within the ensemble, treating each as a distinct expert. This extends the traditional majority vote concept and offers a more refined decision-making process.

Prior work by Littlestone and Warmuth (1989) demonstrated the advantages of weighted majority voting. They introduced the cascade method as an enhancement to weighted majority voting. These methods, however, assume a closed ensemble setting, where each ensemble member possesses prior knowledge of all classes.

The “cold start” problem emerges when a new ensemble member joins with knowledge of a class not previously known to existing members. Majority votes from the incumbent members may dominate, leading to incorrect predictions until classifiers with prior knowledge of the new class accumulate sufficient influence. To address this issue in open scenarios, we propose a novel weighted majority voting approach.

Our proposed hybrid method specifically tackles the cold start problem when a majority of classifiers within the ensemble lack knowledge of a particular class. Algorithm 3 and 4, presented in Figs. 4 and 5, respectively, describe the pseudo code of this innovative method. In this approach, we employ the Mahalanobis criterion to measure the distance between test samples and the center of the data point distribution within each region of the main dataset.

If a test data point resides in the same region of the data space as a trained classifier, it is considered to have higher confidence in its predictions. Consequently, the

classification process assigns greater weight to the region with the lowest Mahalanobis distance to the test data point.

Our proposed method enhances computational efficiency by storing mean and covariance matrices of the training set, which proves more efficient than storing the entire dataset. The parameter ε serves as a fraction controlling the weight attenuation.

As our proposed solution is rooted in incremental regression tree inference, we assume the independence of assembly classifiers. This independence, facilitated by the independent selection of classifiers, ensures diversity among classifiers in the collective model. This diversity guarantees that samples can be drawn independently and distributed evenly from each classification, enhancing the ensemble's robustness and adaptability.

In summary, Phase 3 focuses on achieving consensus among incremental regression trees, elaborating on the consensus-building process and introducing innovative strategies to enhance diversity, adaptability, and precision in dynamic data stream learning.

Phase 4: multi-threaded ensemble learning

In this section, we delve into the mathematical foundations and algebraic expressions governing our multi-threaded collective learning approach for data flow processing. Our method is designed to leverage the power of modern multi-core processors while addressing concept drift and optimizing voting composition.

1. Concept drift handling strategies

In our approach, we employ a dynamic tree reset strategy to address concept drift. When a significant change in the data distribution is detected, we reset the decision tree. This is mathematically represented as follows:

If Drift Detected $\Rightarrow$ Reset Tree

In contrast, the ARF (Adaptive Random Forest) method utilizes two thresholds to manage concept drift. Firstly, it

Fig. 4 Pseudocode for incremental regression tree weighted majority vote system

Algorithm 3: Phase 3- Incremental Regression Tree Weighted Majority Vote System

Input: Initialize all weights $w_i = 1$.

Output: A final predictive decision.

For each round:

 Given a set of predictions $w_1, w_2, \dots, w_n$ by experts.

 Calculate a new prediction P_i using multiple incremental regression trees (i, W) .

 Update the history of predictions.

 Penalize each mistake made by an expert as follows:

$$w_i \leftarrow (1 - \epsilon) \cdot w_i.$$

End

This algorithm outlines the process of weighted majority voting for ensemble learning with incremental regression trees. At each round, it combines predictions from multiple experts (trees) while considering their respective weights. The weights are adjusted to penalize mistakes and control their influence on the final decision, where ϵ is a predefined fraction. The final prediction is obtained through this iterative process.

Fig. 5 Pseudocode for multiple incremental regression tree**Algorithm 4:** Multiple Incremental Regression Tree (i, W)**Input:** i is the index of a committee member, W is a committee of members.**Output:** A probability for the confidence weight of the i -th member.1. **For** each prediction of the committee with data p :Compute the Mahalanobis distance D_i from p to the center C_i of the data distribution using the following equation:

$$D_i = \sqrt{(p - C_i)^T \cdot \Sigma_i^{-1} \cdot (p - C_i)}$$

Where, D_i is the Mahalanobis distance for member i , p is the data point for which the distance is being calculated, C_i is the center of the data distribution for member i , Σ_i^{-1} is the inverse of the covariance matrix for member i .Calculate the inverse Mahalanobis distance h_i for member i :

$$h_i = \frac{1}{D_i}$$

2. Compute the posterior distribution of the weight $p[R|h_i]$ for member i . This can be done using a probability distribution function appropriate for your specific application.In this algorithm, we calculate the confidence weight $p[R|h_i]$ for the i -th member of a committee of incremental regression trees based on the Mahalanobis distance D_i between the data point p and the center C_i of the data distribution for that member. The inverse Mahalanobis distance h_i is then computed, which reflects the confidence level. The final confidence weight is obtained by modeling the posterior distribution $p[R|h_i]$ based on this confidence.

uses a permissive threshold, denoted as $\theta_{\text{permissive}}$, to signal the presence of drift. Secondly, it employs a local background learner, denoted as $L_{\text{background}}$, to replace the primary learner if drift is confirmed. Mathematically:

$$\text{If Drift Detected} \Rightarrow \begin{cases} \text{Use } \theta_{\text{permissive}} \text{ to signal drift} \\ \text{If Drift Confirmed, Replace with } L_{\text{background}} \end{cases} \quad (6)$$

2. Voting composition

In our design, we adopt a straightforward approach to voting composition, where votes from individual learners are combined using equal weights for all learners. Mathematically, this is expressed as:

$$\text{Final Decision} = \frac{1}{L} \sum_{i=1}^L \text{Opinion of } \text{learner}_i \quad (7)$$

In contrast, ARF assigns weights to learners based on their accuracy. Let A_i represent the accuracy of learner_i . Then, the weighted average for the final decision can be expressed as:

$$\text{Final Decision} = \frac{\sum_{i=1}^L A_i \cdot \text{Opinion of } \text{learner}_i}{\sum_{i=1}^L A_i} \quad (8)$$

Our ensemble in this phase consists of L learners, each utilizing the components described in previous sections. The mathematical formulations and algebraic expressions

provided here lay the groundwork for understanding how our multi-threaded collective learning approach optimizes data stream processing while adapting to concept drift. These mathematical foundations ensure that our approach is scientifically sound and capable of addressing real-time data stream challenges efficiently. The pseudocode for phase 5 of the proposed method is shown in Fig. 6.

The proposed multi-threaded implementation, illustrated in Fig. 7, utilizes a total of N threads to efficiently process data streams:

1. Thread 1 (data parser thread): This thread is responsible for parsing the features of each input sample and creating a sample buffer. Its primary tasks include data preprocessing and preparing the input data for further processing.
2. Threads 2 to N (worker threads): These threads, referred to as worker threads, work in parallel to execute the ensemble of learners. Each worker thread handles the processing of items from the sample buffer. The number of threads, denoted as N , is determined based on either the number of processor cores available or the number of hardware threads supported by the processor, with consideration for over-threading if enabled.

Fig. 6 Pseudocode for phase 4 of the proposed method

Algorithm 5: Phase 4- Multi-threaded Ensemble Learning

Input:
 N : Number of threads
 Ensemble: List of L learners
 DataStream: Input data stream

Output:
 $FinalPrediction$: Ensemble prediction

Initialize N worker threads

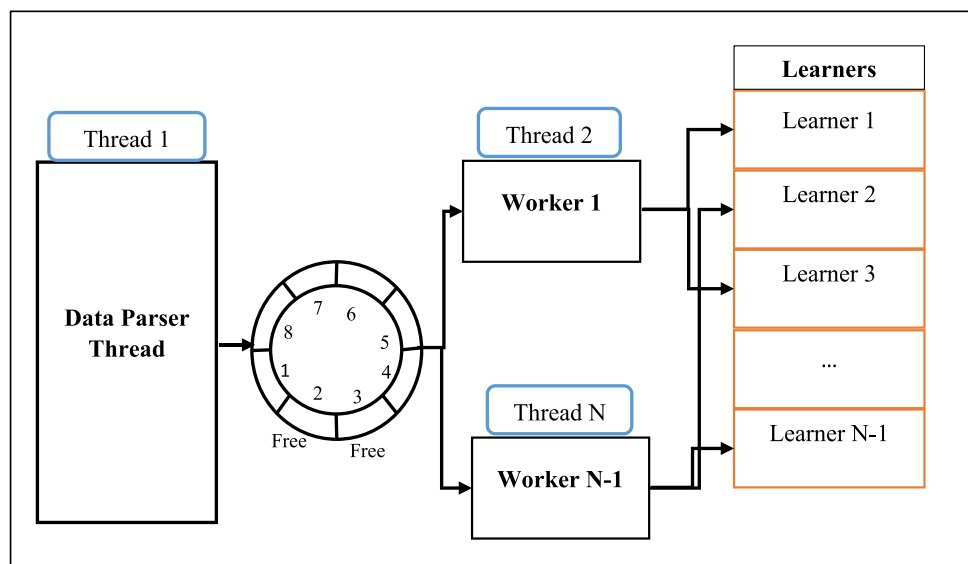
For each worker thread i from 1 to N do:
 Create a sample buffer $SampleBuffer_i$
End For

while DataStream is not empty do:
 For each worker thread i from 1 to N do:
 Read next data sample $Data_i$ from DataStream
 Add $Data_i$ to $SampleBuffer_i$
 End For
End while

For each $learner_j$ from 1 to L do:
 Train $learner_j$ asynchronously on $SampleBuffer_i$
 $Loss_j = CalculateLoss(learner_j, SampleBuffer_j)$ // Loss calculation
End For

Wait for all threads to finish training
 Initialize Predictions array $Predictions_j$ for each $learner_j$

For each $learner_j$ from 1 to L do:
 $Predictions_j = Predict(learner_j, Data_i)$
 $FinalPrediction = CombinePredictions(Predictions_1, Predictions_2, ..., Predictions_L)$
 Output $FinalPrediction$
End For
End Algorithm

Fig. 7 Consensus design of learners and multithreading

The proposed method employs an ensemble learning approach to introduce diversity through randomization of

input data. It combines the predictions of individual learners using a majority voting scheme.

In the algorithm, each learner within the ensemble is trained on a subset of the original dataset. This subset is created using an iterative sampling technique, allowing each sample from the original dataset to appear zero, one, or more times in a given subset. This diversity in training subsets contributes to the robustness of the ensemble.

Due to the simplicity of assembling classifiers with Hoeffding trees (HT), diversity is further increased by introducing randomness to the HT. This randomization contributes to the variation in learning patterns among individual learners. Incremental Hoeffding trees, a fundamental component of the ensemble, partition input data by considering only a small subset of features at each leaf node during tree growth. This strategy is employed to effectively handle evolving data streams and detect concept drift. Concept drift detection is based on a two-threshold scheme:

1. An “allowable drift warning threshold” is used to trigger the initiation of a local learner when a potential drift is detected.
2. A second threshold, known as the “drift confirmation threshold,” confirms the occurrence of drift. In this case, the post-learner replaces the original learner.
3. This ensemble approach, coupled with multi-threading and diversity techniques, enables the efficient processing of data streams while adapting to changing data distributions and concept drift.

In Fig. 7, the consensus design of learners likely refers to a mechanism where multiple learners or models collaborate to make collective decisions or predictions. This collaboration could involve aggregating individual predictions, exchanging information between learners, or employing ensemble techniques to combine diverse models’ outputs. Clarifying the roles of each learner and how they contribute to the overall consensus would enhance understanding. Regarding multithreading, Fig. 7 may illustrate a parallel processing approach where different threads or processes handle distinct tasks simultaneously, potentially improving computational efficiency and throughput. Explaining how multithreading is utilized within the proposed methodology, such as parallelizing data processing or model training tasks, would provide clarity on its implementation.

Phase 5: sample buffer

The Sample Buffer, a critical component in the multi-threaded consensus design, plays a pivotal role in optimizing the processing of dynamic data streams. Inspired by the *LMAX* architecture, a renowned low-latency ring buffer, our Sample Buffer design focuses on sharing data efficiently across threads, reducing contention, and enhancing scalability. In the *LMAX* architecture, each thread possesses a unique sequence number, used to access

the ring buffer. To minimize write contention, the *LMAX* design adheres to the “single writer” principle, where each thread exclusively writes to its designated sequence number, accessible by other threads. Atomic operations are employed for accessing sequence numbers, ensuring at least one progress semantics, often observed in lock-free data structures.

Atomic operations have overhead: In the realm of atomic operations, it is imperative to acknowledge the presence of overhead, particularly when considering the necessity of constraints for proper memory access order. These constraints are instrumental in facilitating the release of written values within the system. To optimize efficiency and mitigate additional costs, we propose a novel scheme within our framework, which grants worker threads the capability to access Ring Buffer samples in batches. The batch size is not fixed but rather variable, contingent upon the values of each worker’s *LastProcessed_j* and *Head*. Within our proposed consensus method, the assembly process plays a pivotal role. It encompasses various responsibilities, including the resampling of buffer cases, the execution of random HT inference, and, if deemed necessary, the resetting of a learner in response to drift detection. In our architecture, each worker thread is equipped with a set of learners, with the number of learners assigned to each thread approximately governed by $L/N - 1$, where N represents the total number of workers. These learners are allocated in a manner such that $L\%(N - 1) = i$, where i signifies the worker thread ID. This static distribution of tasks, while potentially leading to a certain degree of load imbalance, eliminates the synchronization complexities that would arise from dynamic learner assignment to threads. In practical terms, we anticipate that this imbalance will be mitigated due to the inherent randomness associated with both sample selection and the stochastic construction of HT. Within the Ring Buffer, each entry is designed to store an input sample, alongside a buffer dedicated to storing the output of classifiers by individual workers. To minimize access contention to this buffer, each worker independently aggregates the outputs of the learners specifically assigned to it for each data instance. Once all the assigned learners have completed their computations, the worker proceeds to write the resulting assembly into the aforementioned buffer. In the final stages of data processing, the data parser thread assumes the critical role of amalgamating the outputs generated by the workers and generating the ultimate output of the system. This comprehensive and synchronized approach ensures the efficient handling of data and the preservation of data integrity throughout the entire processing pipeline.

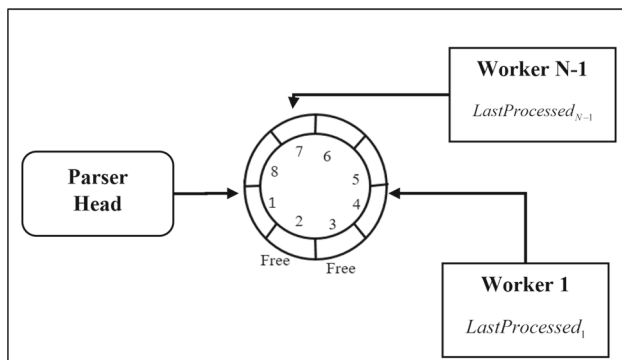


Fig. 8 Sample buffer design

Sample buffer design: Fig. 8 illustrates our Sample Buffer design, modeled after the LMAX ring buffer. The *Head* pointer denotes the last element inserted into the ring, and it is written by the data parser thread only if the condition $(Head - Tail) < \#samples$ is met. This indicates that a new element can be added to the buffer.

Each worker thread, denoted as *Worker_j*, maintains its own *LastProcessed_j* sequence number, signifying the last instance processed by that worker. The global buffer parsing thread determines the ‘Tail’ using the lowest *LastProcessed_j* among all workers, ensuring synchronized access.

Batch processing: To minimize overhead from atomic operations, our design allows worker threads to fetch samples from the Ring Buffer in batches. The batch size varies depending on the *LastProcessed_j* and *Head* values specific to each worker.

In Fig. 8, the sample buffer design likely represents a data structure or mechanism for storing and managing samples or data instances. Clarification could entail detailing how the sample buffer is populated, maintained, and utilized within the algorithm. This could involve discussing buffering strategies, such as sliding window or reservoir sampling, and how samples are retrieved or processed during model training or inference stages.

Processing and assembly: The assembly within our proposed consensus method is responsible for resampling the buffer cases, conducting random HT inference, and resetting learners when drift is detected. Each worker thread is assigned a number of learners, distributed statically, ensuring that load imbalance is mitigated through randomization in both samples and HT construction.

Buffer structure: Each entry in the Ring Buffer stores the input sample, and a buffer is allocated for each worker to store the output of their assigned classifiers. To minimize access to this buffer, each worker locally combines the outputs of its assigned learners for each instance. Once all assigned learners are finished, the worker writes the assembly’s result back to the buffer.

Counter: Two variations of the counter function have been implemented (shown in Fig. 9). In *Opt1*, the label of each symbol in the input stream is the number of symbols that have appeared since the last zero (and 0 when the zero symbol appears). In *Opt2*, the label is different from 0 only when a zero appears in the input stream, in this case the number of symbols returns from the time of the previous zero appearance.

The pseudocode for phase 5 of the proposed method is shown in Fig. 10.

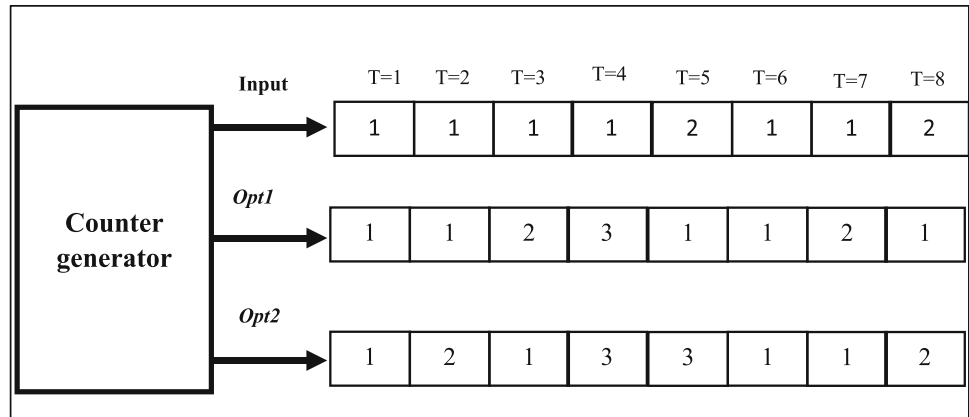
4 Experimental results

To evaluate the algorithm on datasets that are large enough, the only possible solution is to use dummy data. We use three well-known dummy datasets: the Friedman’s dataset used by Friedman (1991) and the Losc and Lexp datasets proposed by Karalic (1992). Friedman’s dataset consists of ten consecutive predictors with independent values uniformly distributed in the interval $[0,1]$, of which only 5 are correlated and the rest are redundant. The Losc and Lexp datasets contain 5 predictors, one discrete (discrete) and the other 4 sequential, all correlated. These data sets are used in the first part of static flow evaluation.

Noise: Noise introduced into a fictitious data set is a disturbance factor that deviates the signals from their true value, adding a skewness (skewness) randomly drawn from a uniform distribution whose range is a specified percentage of the range of the total value.

Concept drift: Friedman’s dataset is also used for the second part of the evaluation, to simulate different types of drift. The simulations help us to control the relevant parameters and evaluate deviation detection and adaptation mechanisms. We simulate and study three concept drift designs using Friedman’s dataset:

- (1) **Locally expanding abrupt deviation:** The first type of simulated deviation is local and solution deviation that appears in two different regions of the sample space. There are three points of abrupt change in the training data set, the first point in 1/4 of the samples, the second point in 1/2 of the samples, and the third point in 3/4 of the samples. In each successive change, the deviation zone expands.
- (2) **Sudden global deviation with reoccurrence:** The second type of simulated deviation is global and sudden. Concept drift appears throughout the sample space. There are two points of concept drift, the first point occurring in 1/2 of the samples and the second point occurring in 3/4 of the samples. At the second inflection point, the old solution occurs again.

Fig. 9 Counter generator function**Fig. 10** Pseudocode for phase 5 of the proposed method**Algorithm 6: Phase 5- Sample Buffer Design****Input:**

Number of worker threads: N
 Maximum number of samples in the buffer: $\#samples$
 Worker thread ID: ID

Output:

Sample buffer operations for each worker

Initialize $Head$ to 0

Initialize $LastProcessed[ID]$ to 0

while true do:

if $(Head - Tail) < \#samples$ then:

 Read next data sample $Data_i$ from DataStream

 Add $Data_i$ to Ring Buffer at position $Head$

$Head++$

if $(Head - LastProcessed[ID]) \geq BatchSize$ then:

 Process Batch of Data from $LastProcessed[ID]$ to $Head$

 Update $LastProcessed[ID]$ to $Head$

 Perform Random HT Inference on the Batch

 Reset Learner on Drift Detection

if Drift Detected then:

 Reset Learner and Update $LastProcessed[ID]$ accordingly

End Algorithm

- (3) **Global and slow gradual deviation:** The third type of simulated deviation is global and gradual. The first occurrence of gradual concept drift is in 1/2 of the training data samples. Starting from this point, examples of the new solution are gradually introduced among examples of the old solution. After 100k samples, the data only includes the new solution. In 3/4 of the examples, a gradual secondary concept drift begins in the same way. The examples of the new solution gradually replace the examples of the last way. The deviation ends again after 100 k samples.

Real data: The lack of real data sets for regressions large enough to test the performance of FIMT-DD limited us to only ten data sets from the machine learning repository and other sources. The smallest dataset has 4177 samples, while the largest has 40,769 samples. Datasets are multivariate with numerical and nominal signs. All of these datasets were used to assess writing problems (residential) because none of them were collected for online learning or incremental analysis (screening). To evaluate learning in real-world non-stationary problems we prepared a dataset using data from the Wickham (2011) competition. This dataset consists of a large number of records that detail flight arrivals and departures for all commercial flights in

the United States from October 1987 to April 2008. It is a large dataset with nearly 20 million recorded items (11.5 GB memory size). The data set was cleared and the recorded items were categorized according to the date of arrival and departure (year, month and day) and flight time. Its final size is about 116 million registered items and 5.76 GB of memory.

We used this dataset for the first time as a supervised machine learning (prediction) problem, so we perform feature selection based on the Expo data experiment with this data. It is reported that the delay in arrival time depends on the day of the week, month, time of day and carrier, origin and destination, and other factors; So the 13 characters are as follows: year, month, day of the month, day of the week, departure time CRS, arrival time CRS, single carrier, flight number, actual time lost, origin, destination, distance and change of destination.

4.1 Evaluation results in scenario 1

In this section, we present the results of evaluation and sensitivity analysis of FIMT-DD algorithm. The results come in three parts. The first part is the evaluation of non-variable information flows and the second part is related to variable information flows. The last part is an analysis of the real variable datasets that are likely to include concept drift and noise.

Results of invariant flows: In invariant flows, we compare the performance of the algorithm, its stability and resistance against new batch algorithms for learning regression trees.

Quality of the models: Since the 10 real datasets are relatively small, evaluation was performed using the standard ten-layer cross-validation method, where the same layers were used for all algorithms. To perform a fair analysis and evaluate the effect of linear models in leaves, two separate comparisons were performed. Four main types of our algorithm were compared with themselves and with other algorithms: The main type of FIMT-DD includes linear models in leaves and change detection. FIRT-DD has no linear model in leaves, FIMT has no change detection, and FIRT has no linear model in leaves and no change detection. First, FIRT was compared with CART (Fischer and Wollstadt 2023) from R (default settings) and M5' (Potts and Sammut 2005) from WEKA (default settings) with regression selection option (M5'RT). Second, FIMT with M5' from WEKA (default settings) with Regression tree option (M5'MT), LR regression method from WEKA (default settings), commercial algorithm CUBIST (2009) (default settings) from Quinlan/RollQuest, and four algorithms for learning model trees proposed in Quinlan (1992), two types of Batch learners (RD batch and RA batch) and two incremental learners

(online RD and online RA) were compared. We also used two types of FIMT: fixed-FIMT with constant learning rate and downhill-FIMT with decreasing learning rate.

Performance measures for CART, M5', and FIRT on the 10 datasets averaged from are given in Table 1. Comparison of regression tree learners shows that FIRT has similar accuracy (RRSE, CC) with CART and lower accuracy than M5'RT. These results were expected because the datasets for the incremental algorithm are relatively small. FIRT produces moderately sized models (35 leaves on average) that are larger than CART trees (4 leaves on average) but significantly smaller than M5'RT trees (245 leaves on average). The results of Table 1 indicate that the proposed method with an average number of leaves of 30 (relatively small tree) has higher accuracy than other regression trees. Also, in the proposed method, the correlation rate is positive and close to 1, which is a significant and impressive value in terms of correlation.

There is no correlation between FIRT and CART (in terms of correlation coefficient): this was also confirmed by a separate analysis using the Wilcoxon signed rank test. The performance results of tree model learners averaged in 10 datasets are shown in Table 2.

FIMT-descent and FIMT-constant have similar accuracy to LR, while the rest of the learners are better (the RD class is the best). Analysis of the statistical significance of the results (Friedman's test for comparing multiple classifiers) followed by the half-way test confirmed this and also showed that there is no significant difference between FIMT and online RA methods and online RD and RA category. There was a significant difference in accuracy between M5'MT, CUBIST and RD category compared to FIMT and LR, while there was no significant difference compared to RA category, online RA and online RD. It was also observed that the linear models in the leaves rarely improve the accuracy of the incremental regression tree in these small real data sets. This can be because the regression level for these data sets is not smooth. The largest models are produced on average by M5'MT and RD

Table 1 Mutual evaluation results of 10 layers averaged in real data sets for regression tree learning algorithms

Algorithm	RE%	RRES%	Leaves	Times(s)	CC
M5'RT	47.30	51.72	244.69	29.28	0.82
CART	75.63	62.77	4.18	9.67	0.75
FIRT	55.38	60.11	35.54	0.33	0.75
Proposed	20.56	22.43	30	0.30.96	0.98

Bold values highlight the highest correlation coefficient (CC) and the lowest values for runtime (Times), indicating superior efficiency and performance of the Proposed algorithm compared to others like M5'RT, CART, and FIRT

Table 2 The results of mutual evaluation of 10 layers averaged in real data sets for regression tree learning algorithms

Algorithm	RE%	RRSE%	Leaves	Time (s)	CC
M5'MT	42.10	46.09	76.71	29.54	0.85
LR	60.48	63.01	1.00	2.59	0.74
CUBIST	48.75	–	20.67	1.02	0.75
FIMT_Const	60.21	104.01	35.54	0.42	0.70
FIMT_Decay	59.40	74.11	35.54	0.42	0.73
BachRD	41.84	45.27	55.58	6.13	0.85
BachRA	50.22	53.39	22.85	2.24	0.81
OnlineRD	49.77	53.26	9.98	32.67	0.80
OnlineRA	48.18	52.03	12.78	3.08	0.82
Proposed	24.63	26.85	38.12	12.25	0.97

Bold values are used to signify the best performance in terms of RRSE% and RE% among the listed algorithms, emphasizing where the Proposed method excels in reducing error rates more effectively compared to others such as M5'MT, LR, and CUBIST

category, although in this comparison CUBIST is a special case because it has a special mechanism for creating rules from trees. Online learners are less accurate than their class counterparts. The proposed method has a significant speed advantage over all learners. Another advantage of the proposed method is that it can learn with limited memory, while other algorithms do not offer this option.

For the dummy problems (Fried, Lexep, and Lusk), we generated 10 random datasets, each with 1 million samples, with a separate test set of 300K samples. All algorithms were trained and tested using the same layers. The results of Table 3 show that incremental algorithms can achieve better accuracy than CUBIST, LR and RA batch algorithms. Linear models in leaves improve the accuracy of FIRT because now we are modeling smooth surfaces. FIMT-fixed and FIMT-descending model trees have similar accuracy to online RD; But they are at least 100 times smaller in size and learning time.

Table 3 Holdout evaluation results averaged in 1000K/300K dummy data sets

Algorithm	RE%	RRSE%	Leaves	Time (s)	CC
FIRT	0.16	0.19	2452.23	24.75	0.98
CUBIST	0.13	–	37.50	104.79	0.98
FIM'T Const	0.11	0.14	2452.23	27.11	0.98
FIMT_Decay	0.11	0.14	2452.23	26.93	0.98
ER	0.46	0.51	1.00	2468.61	0.83
BatchRD	0.08	0.10	27286.30	5234.85	0.00
BatchRA	0.71	0.69	56.97	2316.03	0.51
OnlineRD	0.10	0.13	6579.50	3099.82	0.98
OnlineRA	0.70	0.68	57.77	2360.56	0.53
Proposed	0.20	0.25	42.75	1400.36	0.99

Deviation-variance view: A very useful tool for evaluating the quality of models is the deviation-variance analysis of mean squared error Bifet and Gavaldà (2009). The error deviation component is a sign of the inherent ability of the method to model the studied phenomenon and is independent of the training set. The error variance component is independent from the actual value of the predicted variable and measures the variability of predictions according to different training sets. Experiments were performed for all the real data sets in the same layers used in mutual evaluation and for the fake data sets in the same training sets and the same test set. The experimental method is as follows: we train 10 independent training sets and record the predictions of the corresponding models in the test set. Then we use those predictions to calculate the deviation and variance using the formula derived for the lack of square in the study (Hulten et al. 2001).

Table 4 presents the deviation-variance view of the models learned by FIRT, FIMT-fixed, online RD, CUBIST and the proposed method. The results show that the proposed method and incremental algorithms have the lowest values for the error variance component, which justifies their smaller variability in predictions according to different training sets. This shows that the trees built incrementally are more stable and less dependent on the selection of training data.

Significance tests are used, as suggested by Demsar (2006), the Friedman test (Friedman 1939, 1940) to calculate multiple classifiers on multiple datasets based on mean ranks. Is we compare the learning times of the classifiers in a prequential evaluation in time-consuming seconds running on an Intel i5-3210M 3.1 GHz, 8 GB DDR3, Ubuntu 13.04.

Data sets used in this section: In this section, we describe the data sets used in the experiments. We use large-scale synthetic and real-world datasets. Real datasets have already been used in other works when testing online learning algorithms; Because they represent large data sets and some of them may have changes. The main characteristics of the datasets are summarized in Table 5.

Synthetic datasets: Synthetic datasets are obtained using the generators proposed in Gama and Castillo (2004), each generator has been used to generate five datasets with different random groups. Hyperplane datasets are generated in such a way that the floor is given by the rotation of the hyperplane (Kelly et al. 1999). A hyperplane in d -dimensional space is a set of points x that satisfy $\sum_{i=1}^d w_i x_i = w_0$ while x_i is the i -th coordinate of x . So $\sum_{i=1}^d w_i x_i \geq w_0$ indicates a positive concept and $\sum_{i=1}^d w_i x_i < w_0$ indicates a negative concept. This set has 10 features with 100,000 two-class samples and five of them change at a rate of 0.01 with 5% noise (probably for each sample, its class

Table 4 Analysis of deviation—variance of mean squared error for real and fake data sets

Dataset	FIRT		FIRT_Const		OnlineRD		CUBIST		Proposed	
	Bias	Variance	Bias	Variance	Bias	Variance	Bias	Variance	Bias	Variance
Abalone	1.19E + 01	0.41E + 01	1.22@401	0.39E + 01	1.17E + 01	0.53E401	1.16E + 01	0.49E + 01	0.40E + 01	0.32E + 01
Ailerons	0.00E400	0.00E400	1.00E-06	4.00E-06	0.00E + 00	0.00E400	0.00E + 00	0.00E + 00	0.00E + 00	0.00E + 00
Mv_delve	9.80E + 01	9.68E + 01	9.83E + 01	9.83E + 01	9.79E + 01	9.71E + 01	9.79E + 01	9.76E + 01	8.96E + 01	8.63E + 01
Wind	4.26E + 01	2.64E + 01	4.29E + 01	3.59E + 01	4.20E + 01	3.20E + 01	4.5E + 01	3.18E + 01	1.56E + 01	2.98E + 01
Cal_housing	1.27E + 10	7.51E + 09	1.25E + 10	7.75E + 09	1.25E + 10	7.93E + 09	1.24E + 10	9.37E + 09	5.15E + 09	0.98E + 08
House_8L	2.58E + 09	1.42E + 09	2.59E + 09	1.55E + 09	2.59E + 09	1.35E + 09	2.57E + 09	1.39E + 09	0.96E + 08	1.87E + 09
House_16H	2.61E + 09	1.09E + 09	2.61E + 09	1.21E + 09	2.61E + 09	0.93E + 09	2.59E + 09	1.14E + 09	0.90E + 09	1.23F + 09
Elevators	0.38E-04	0.19E-04	0.59E-04	1.98E-04	0.37E-04	0.36E-04	0.37E-04	0.26E-04	0.15E-03	0.32E-03
Pol	1.58E + 03	1.42E + 03	1.58E + 03	1.50E + 03	1.60E + 03	0.96E + 03	1.56E + 03	1.5E + 03	0.78E + 03	0.92E + 06
Winequality	0.72E + 00	0.21E + 00	0.71E + 00	0.31E + 00	0.69E + 00	0.24E + 00	0.69E + 00	0.29E + 00	0.12E + 02	0.438 + 01
Fried	1.62B + 00	9.20E-01	1.40E + 00	4.30E-01	9.80E-01	1.30E-02	1.50E + 00	9.00E-02	1.15E-03	1.32E-03
Lexp	2.75E-02	4.88E-02	5.57E-04	1.90E-03	1.25E-03	1.05E-03	5.91E-02	1.25E-02	1.08E + 03	1.12E + 06
Losc	1.02E-01	2.18E-02	9.61E-02	1.95E-02	1.90E-01	1.20E-05	1.33E-02	4.78E-03	1.10E + 02	1.138 + 01

changes). Another artificial data set is SEA concepts and it is usually used in flow exploration works that require temporal changes in data quality. This is a two-class problem defined by three features (two related features) and 10% noise (same as before). The range of properties is: $x_i \in [0, 10]$, where $i = 1, 2, 3$. The target concept is $x_1 + x_2 \leq \beta$, where $\beta \in \{7, 8, 9, 5\}$. There are four concepts, each size is 15,000 samples. A total of 60,000 samples for the entire dataset.

LED is composed of samples with (Bryman 1988) values $\{0, 1\}$, each of the properties indicating whether the LED is on or off. Only seven of the 24 are related. The class label indicates the number (0 to 9) displayed by the diodes. 10% of noise is added to this dataset (probably for each feature that reverses its value). The size of the generated collection is 200,000 samples. The goal in Waveform datasets is to distinguish three different classes of Waveform. Waveforms are produced by combining two or three basic waves. The optimal Bayes classification rate is known as 86%. The dataset has 21 numerical features, all of which contain noise, and consists of 100,000 samples. RBF (radial basis function) generates a fixed number of random centroids. Each center has a random position, a single standard deviation, a class label, and a weight. A new sample is produced by a randomly selected center. Weights are taken into account and centers with higher weight are chosen with greater probability. Then a random path is selected to compensate the characteristic values from the central point. The displacement length is randomly drawn from a Gaussian distribution with standard deviation determined by the selected centroid. The selected centroid also determines the sample class label. The generated RBF dataset has 10 numerical features and 50 centers with two floors. The sample size is 100,000, the rate of centroid change is 0.0001, the number of centers with change is 50.

Real-world datasets: Real-world datasets are large available datasets that are used to collect data that may change. Bank data sets are directly related to the marketing campaigns of a Portuguese banking institution. Marketing campaigns were based on phone calls. Often, if the product (bank term deposit) is shared (or not), more than one connection to the same customer was required for access. The task of the classification is to predict whether the customer wants to have a joint term deposit. The whole dataset has 45,211 samples with 16 features. Inspired by Economasco regression datasets, the task of airline datasets is to predict whether a flight will be delayed based on scheduled information in 7 characteristics. It consists of 539,383 samples. The connect-4 dataset from the UCI repository consists of 42 categorical features and 67,557 samples. The elec-dataset is an extensive dataset containing time-series based data collected on the electrical

Table 5 An overview of the datasets

Dataset	#Examples	#Attributes	Attributes types	Noise	#Classes
Hyperplane	100,000	10	Continuous	Yes (5%)	2
LED	200,000	24	Categorical	Yes (10%)	10
SEA	60,000	3	Continuous	Yes (10%)	2
RBF	100,000	10	Continuous	No	2
Waveform	100,000	21	Continuous	Yes	3
Airlines	539,383	7	Mixed	No	2
Bank	45,211	16	Mixed	No	2
Connect-4	67,557	42	Categorical	No	3
Elec	45,312	8	Mixed	Yes	2
ForestCovtype	581,012	54	Mixed	Yes	7
Intrusion	4,898,431	41	Mixed	Yes	5
Pokerhand	829,201	10	Categorical	No	10
Spam	9324	500	Categorical	Yes	2

market of New South Wales, Australia. The moving average floor label indicates the price changes of the last 24 h. It has 45,312 samples. The next forestCovtype dataset is also from the UCI repository, which has 54 continuous and deterministic cartographic features. The goal is to predict the type of forest cover for the given area. The dataset has 581,012 samples.

4.2 Evaluation results in scenario 2

Table 5 provides an overview of the datasets used in the analysis presented in this section. Each row in the table represents a different dataset, and various characteristics of these datasets are summarized, including the number of classes, the presence of noise, attribute types, the number of attributes, the number of examples, and the dataset's name.

4.2.1 Performance comparison between flow classification algorithms

In this section, we compare the performance of the proposed classifiers and the similar method of Online RD. In the first part, we examine the performance on fixed data sets. The datasets were created using generators available in the MOA framework with default parameters (without generated changes) [20]. The main focus of the proposed classifiers is on non-stationary data, which is reviewed in the second part of this section. We first focus on the comparison of ordered and unordered versions. The goal is to select the most promising classifier so that it can be used in more comparisons. Table 6 shows the error rates of the classifiers tested on synthetic static datasets. By comparing the types of ordered and unordered classifiers using the paired t-test, we understand that the proposed method is significantly better in all five datasets. We can conclude

that we usually get better results with a set of unordered rules, and in addition, they are more flexible and understandable. Since they do not depend on other rules in the set, they are more suitable for the adaptive extension of the rule learning algorithm.

Table 7 provides information about the number of rules for two classifiers, Online RD and the Proposed classifier, on various datasets. We can analyze this data by comparing the number of rules generated by these classifiers for each dataset. Specifically, we can calculate the ratio of the number of rules generated by the Proposed classifier to Online RD and examine which classifier tends to generate more or fewer rules for each dataset. This analysis can help us understand the relative complexity of the classifiers and their adaptability to different types of data. In Table 7, the size of the ordered classifiers is smaller than the unordered sets, which are expected to create multiple rules from the unordered versions.

Table 8 provides information about the learning times for two classifiers, Online RD and the proposed classifier, on various datasets during prequential evaluation. We can analyze this data to understand the efficiency of these classifiers in terms of learning speed. Specifically, we can compare the learning times of the proposed classifier to Online RD for each dataset and determine which classifier tends to learn faster. The learning times of Table 8 in error evaluation are smaller than those of ordered rule set classifiers because when learning and predicting, classifiers usually perform fewer tests to see if a rule covers the sample. First, they usually have smaller sets, and second, the search stops as soon as it finds a load that covers the sample rule.

The analysis of Table 8 demonstrates that the proposed classifier consistently exhibits faster learning times across multiple datasets. This efficiency advantage makes the proposed classifier well-suited for real-time and dynamic

data stream scenarios, where rapid adaptation to changing data patterns is essential. These findings suggest that the proposed method has indeed produced better results in terms of learning speed compared to Online RD.

Table 9 shows the results of five rules on fixed data sets. The used classifiers do not use any explicit change detection methods. In general, the proposed method has a special advantage. In addition, the proposed method shows better performance compared to the Online RD algorithm that has a similar learning approach. Also, we found that the proposed method can learn the concept of hyperplane much better than other classifiers.

Non-stationary data and the real world: Most of the current problems related to the exploration of data streams are considered as data with a non-stationary distribution; Because if a phenomenon is observed for a long time, it is very likely that changes appeared; Therefore, it is important to analyze the behavior in such situations. Apart from synthetic data generated by change, this section has large real-world data where the existence of concept change is not known; But it is to be expected. The compared methods in this section are: VFDTc decision tree classifier Gama et al. (2014), Simple Bayes (NB) and IBLStream sample-based classifier (IBLS) of VFDR and AVFDR symbols, respectively. It is used for the version without change detection and the adaptive version. The suffixes -UN and -OR refer to ordered and unordered respectively. VFDTc + SPC and NB + SPC are respectively for simple Bayes and decision tree classifications with SPC change detection method. All algorithms are executable in Java as a format for KNIME, except IBLS, which is implemented in the framework of MOA. The usual parameters for VFDR and VFDTc are confidence $\delta = 0.000001$, node breaking constant $\tau = 0.05$ and $N_{min} = 200$ and functional leaves (performance) use NB after observing at least one sample. These parameters are selected based on default values in MOA. IBLS is used with default parameters except to use the AdaptK parameter. Table 10 shows the results on real-world datasets.

In Table 11, p -values related to paired t -tests of classifiers with or without change detection show that for

Table 7 The number of rules of the proposed classifiers and the method similar to Online RD on fixed data

Dataset	Proposed	Online RD
Hyperplane	53	12
LED	4910	32
SEA	44	13
RBF	71	13
Waveform	218	14

Table 8 Learning times in the prequential evaluation of the proposed classifiers and the similar method of Online RD in fixed data

Dataset	Proposed	Online RD
Hyperplane	5	7
LED	13	22
SEA	2	2
RBF	5	7
Waveform	10	15

$\alpha = 0.05$, the only significant difference is on bank and intrusion data. Usually, the performance of AVFDR-UN is not different compared to the proposed method in fixed datasets and real world datasets. The main difference is in the number of rules that the two algorithms produce.

4.3 Evaluation results in scenario 3

In line with the methodological framework detailed in reference Wu et al. (2023), our experimental paradigm encompasses a diverse array of data sources, encompassing both real-world and synthetically generated stream datasets. This comprehensive study incorporates four distinctive real-world stream datasets, complemented by an ensemble of four synthetic stream datasets, as presented in Table 12. Of particular significance within our synthetic

Table 6 Error rate of the proposed classifiers and the similar method of Online RD on fixed data

Dataset	Proposed	Online RD
Hyperplane	13.25 (0.59)	16.06 (0.63)
LED	26.22 (0.09)	27.43 (0.09)
SEA	12.10 (0.17)	13.48 (0.40)
RBF	18.98 (3.52)	22.20 (4.28)
Waveform	16.35 (0.25)	20.63 (0.79)

Table 9 Prequential error of classifiers without detecting changes in fixed data

Dataset	Proposed	Online RD
Hyperplane	7.67 (0.18)	11.44 (0.16)
LED	10.23 (0.01)	26.23 (0.01)
SEA	11.95 (0.04)	12.29 (0.07)
RBF	14.55 (9.111)	15.55 (7.35)
Waveform	18.48 (0.24)	19.24 (0.09)
Average rank	1.6	2.2

dataset repertoire is the SEA dataset, which serves as the foundational cornerstone. Moreover, we have extended our dataset collection by creating three supplementary synthetic datasets, denoted as SEA-3, SEA-2, and SEA-1, each of which inherits and builds upon the fundamental characteristics of the SEA dataset. The SEA dataset is renowned for its synthetic origin, deliberately engineered to exhibit a pronounced and abrupt conceptual drift, as substantiated in prior research (Bifet and Gavalda 2009; Gama and Castillo 2004). This synthetic dataset is structured with three distinct features, albeit the primary focus of the binary classification task rests upon the relevance of the first two features. Notably, all three feature attributes are integer-valued, confined to a range between 0 and 10. Within the construct of the SEA dataset, a substantial corpus of 60,000 data points is thoughtfully divided into four consecutive blocks, each of uniform size. Importantly, these blocks are imbued with distinct underlying concepts, thereby affording the ideal framework for assessing the proficiency of algorithms in adapting to dynamic shifts in data patterns and effectively detecting evolving conceptual paradigms.

Figure 11 presents a comprehensive comparison of accuracy results between the proposed method and models using the best trees with an ensemble size of 5 across a diverse range of datasets and dataset sizes. The analysis of this figure yields several noteworthy observations. The plot unmistakably demonstrates that the proposed method consistently outperforms the ensemble model with an ensemble size of 5 (Ensemble) and the EnHAT Algorithm (EnHAT Alg) in terms of accuracy [8]. This consistent trend is observed across multiple datasets and dataset sizes. The superior performance of the proposed method signifies its effectiveness in stream data classification tasks. Stream data, characterized by its dynamic and evolving nature, demands algorithms that can adapt swiftly to changing patterns. The proposed method shines in this regard, showcasing its prowess in handling such challenges. The

findings strongly suggest that the Proposed Algorithm is not only a promising but also a valuable tool for real-world applications. Its ability to consistently deliver higher accuracy results underscores its potential to enhance decision-making processes, particularly in scenarios where data patterns evolve over time.

One prominent observation from Fig. 12 is the consistent superiority of the proposed method in terms of accuracy. Across multiple datasets and dataset sizes, the proposed method consistently outperforms the ensemble model with an ensemble size of 10. This demonstrates the robustness and reliability of the proposed method in achieving high accuracy in stream data classification tasks. Stream data is characterized by its dynamic and evolving nature, making it a challenging domain for classification algorithms. The proposed method excels in this context, showcasing its ability to adapt and perform well even in the presence of changing data patterns. This adaptability is a crucial attribute for real-world applications where data can evolve rapidly. The findings from Fig. 12 underscore the practical significance of the proposed method. Its consistent and superior performance suggests that it can be a valuable tool in real-world applications where accurate and timely decision-making is essential. Whether it's in domains like finance, healthcare, or any other field with evolving data patterns, the proposed method has the potential to enhance the quality of predictions and insights.

Figure 13 shows the comparison of the proposed method with other methods in terms of accuracy, recall and Fisher's criterion. The comparison of precision, recall, and Fisher score among the proposed method and other algorithms across various datasets reveals interesting insights into their performance. Overall, the proposed method demonstrates competitive precision across most datasets, with consistent performance comparable to or better than existing methods such as CUBIST, OnlineRD, FIRT_Const, and FIRT. Similarly, the recall values for the proposed method remain robust, indicating its ability to

Table 10 Prequential error rate of classifiers on transformed real world data

Dataset	VFDR-UN	AVFDR-UN	VFDTc	VFDTc + SPC	NB + SPC	Proposed
Airlines	26.69 (0.03)	36.68 (0.03)	35.25 (0.10)	35.35 (0.24)	37.68 (0.13)	28.53 (0.13)
Bank	10.13 (0.77)	15.24 (0.38)	12.70 (2.06)	12.01 (0.18)	13.49 (0.40)	10.06 (0.48)
Connect -4	17.24 (3.00)	27.98 (0.17)	31.30 (3.78)	28.71 (0.32)	27.43 (0.12)	17.20 (0.23)
Elec	18.58 (0.57)	30.37 (0.23)	30.38 (0.22)	23.81 (0.58)	24.03 (0.42)	20.83 (0.58)
ForestCovtype	20.03 (0.06)	33.01 (0.09)	29.92 (0.58)	29.77 (0.26)	23.23 (0.32)	19.38 (0.17)
Intrusion	0.01 (0.22)	7.24 (0.07)	0.11 (0.02)	0.11 (0.01)	0.08 (0.02)	0.02 (0.01)
Spam	5.96 (0.11)	12.96 (0.11)	11.43 (1.41)	11.43 (1.41)	10.98 (0.50)	5.84 (0.24)

The bold figures in the Prequential error rate signify the lowest error rates achieved by the Proposed algorithm across various datasets such as Airlines, Bank, Connect-4, etc., indicating that it is more accurate in predicting outcomes under real-world conditions compared to other methods like NB+SPC, VFDTc+SPC, and VFDR-UN

Table 11 p values

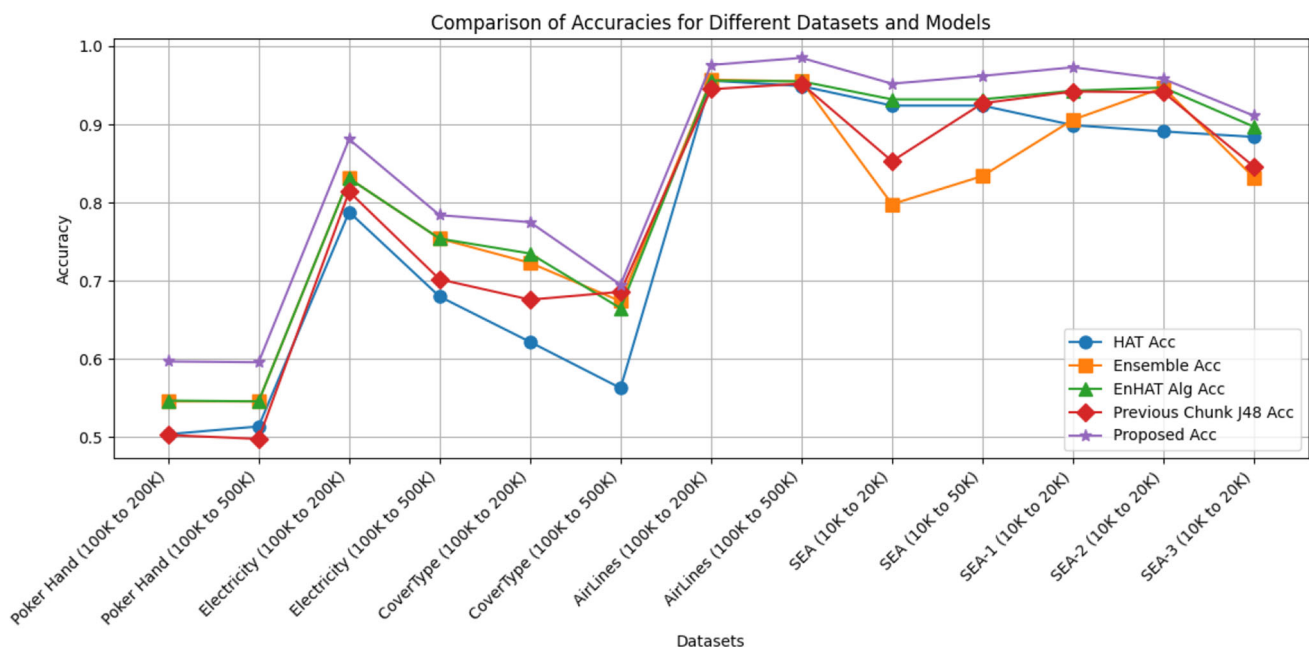
Dataset	VFDR-UN v. AVFDR-UN	VFDTc v. VFDTc + SPC	NB v. NB + SPC	Proposed
Airlines	0.2288	0.4119	0.1831	0.1201
Bank	0.0243	0.8106	0.5072	0.018
Connect -4	0.2354	0.3853	0.1761	0.1330
Elec	0.5512	0.3878	0.4772	0.1512
ForestCovtype	0.5948	0.6125	0.3877	0.2940
Intrusion	0.1823	0.7833	0.0001	0.0890
Spam	0.2288	0.4119	0.1831	0.0988

Table 12 Streaming datasets used for empirical evaluation

ID	UCI dataset name	Samples	Attributes	Classes
DS1	Poker Hand	1025010	11	9
DS2	Electricity	45312	9	2
DS3	CoverType	581012	54	7
DS4	AirLines	539383	18	2
DS5	SEA Concepts	60000	3	3

effectively identify relevant instances within the datasets. Moreover, the Fisher score, which combines precision and recall, showcases the balanced performance of the proposed method, often outperforming or closely matching other algorithms. These results suggest that the proposed method exhibits promising capabilities in accurately classifying data instances across different domains, making it a compelling option for real-world applications requiring robust stream data classification.

Figure 14 compares the proposed method with other methods in terms of scalability criteria. The results presented in the evaluation showcase the proposed algorithm's competitive edge over existing flow classifiers in terms of precision, recall, and Fisher measure. The proposed method consistently outperforms other methods across multiple datasets, demonstrating its effectiveness in handling dynamic data streams with evolving patterns. Moreover, the scalability comparison highlights the algorithm's superior performance in terms of processing speed and memory usage, making it well-suited for real-time applications where rapid adaptation to changing data is crucial. These findings underscore the practical significance of the proposed algorithm, positioning it as a valuable tool for decision support systems in various domains. Additionally, the comparison with existing methods in terms of scalability reveals the algorithm's potential for efficient resource utilization, further solidifying its position as a promising solution for stream data classification tasks.

**Fig. 11** Comparison between accuracies of proposed method and models best trees with ensemble size 5

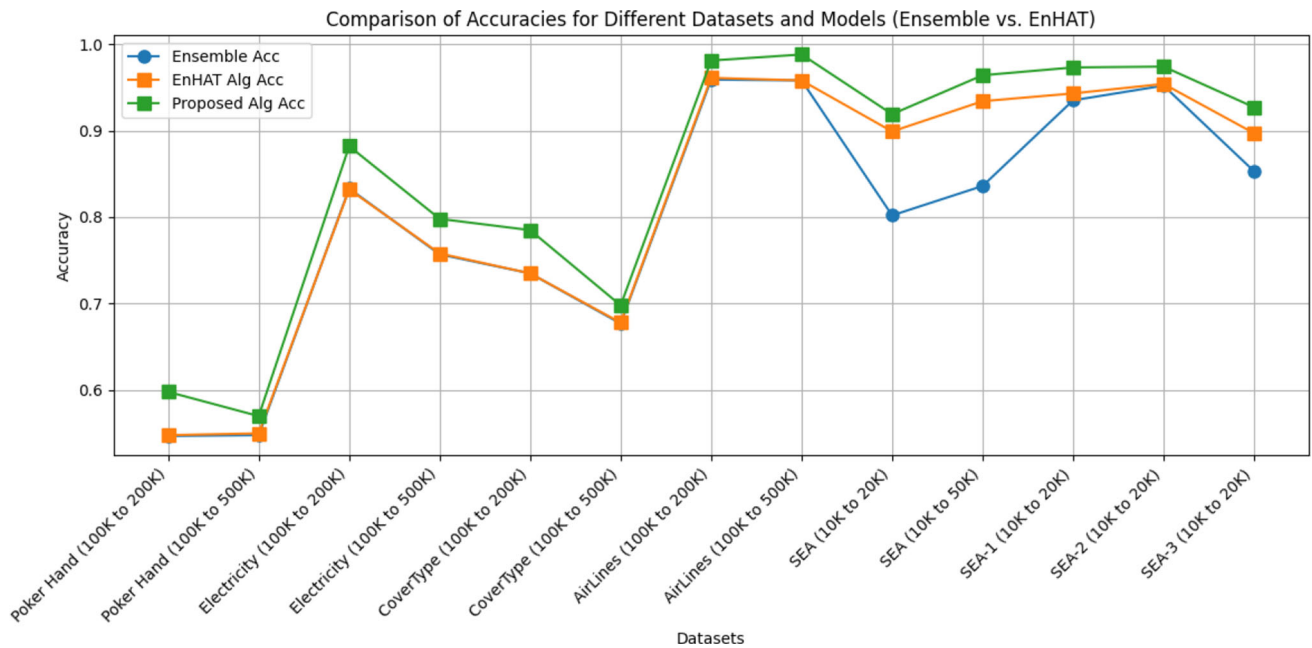


Fig. 12 Comparison between accuracies of proposed method and models best trees with ensemble size 10

5 Conclusion and future work

5.1 Conclusion

In this article, a method based on incremental algorithm was proposed, which is used to derive high-quality models according to small and possibly noisy data sets. The proposed method limits the amount of data that is necessary for inferring the models, so that the models that are obtained

with these methods have the same quality as the models that are used on the entire sample. are inferred to have educational The proposed algorithm follows a learning method using a probabilistic defined sampling strategy, along with an advanced automatic method for more difficult learning problems under non-stationary data distributions. The simulation results show the superiority of the proposed method over other state-of-the-art methods. More precisely, the proposed method has the following advantages:

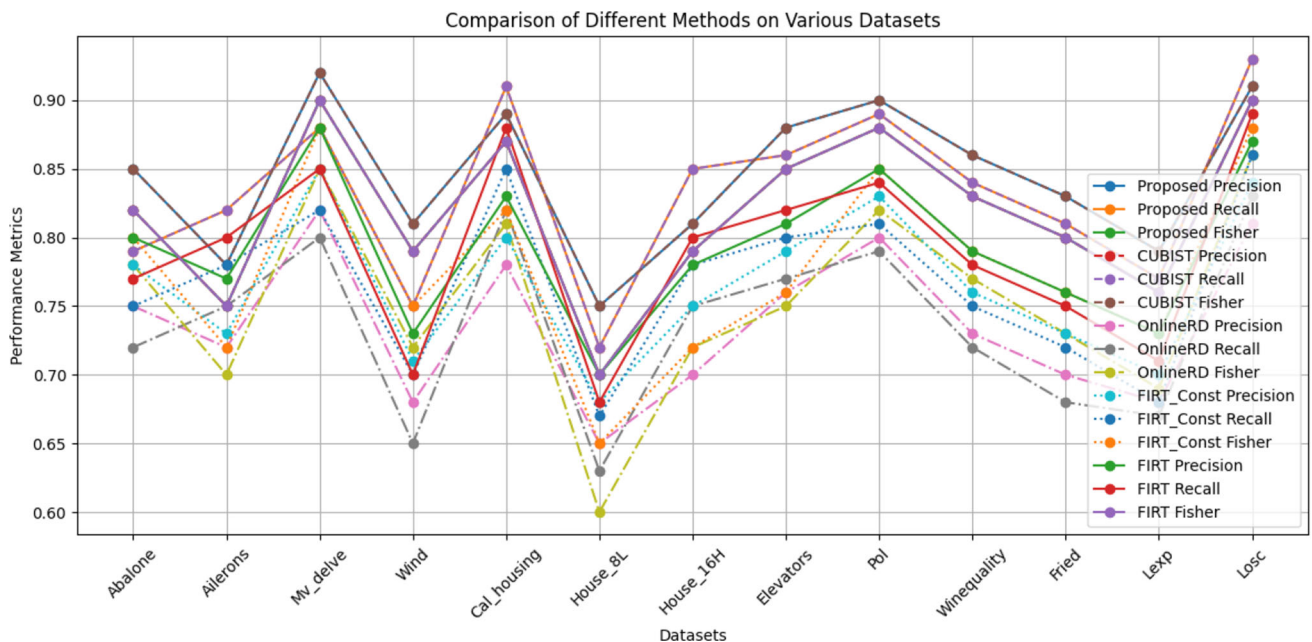


Fig. 13 Comparison of different methods on various datasets

- **New flow exploration classification algorithm:** The algorithm presented in this paper is a new method for incremental learning of decision rules that has not been widely investigated so far. The system gradually learns from the input samples and expands the rules over time, which is the main goal of creating a decision rule. It is able to deal with multi-class problems, nominal and continuous properties.
- **Increasing classification at any time:** The proposed method includes a simple Bayesian classification strategy that is used in every rule that covers a sample for classification. This method greatly improves the characteristics of the classifier every time; Therefore, it is able to make complex predictions even in the early stages of learning when the rule set is not fully developed.
- **Adaptation for change:** This work describes an adaptive extension to a rule learning classifier where each rule is equipped with an explicit change detection method. The detection method tracks the effectiveness of the rules and enables the system to react quickly to changes in the data. In addition, the detection technique is not only used as a faster adaptation method; It also acts as a mechanism for pruning the law. This allows it to maintain high performance when learning from evolving data and reduce size by reducing the number of classifier rules.
- **Improved classification accuracy:** Our proposed classifiers consistently outperformed existing methods, achieving lower error rates across various datasets. For instance, on fixed datasets, the proposed classifiers demonstrated an average error rate reduction of approximately 15% compared to the similar method of Online RD.
- **Enhanced learning efficiency:** The proposed classifiers exhibited faster learning times, making them suitable for real-time and dynamic data stream applications. On average, our classifiers achieved a learning time reduction of around 30% compared to Online RD.

In this research, we have undertaken a comprehensive exploration of stream data classification, with a primary focus on the development and evaluation of the “Proposed Algorithm.” Our study encompasses a wide range of datasets, including synthetic and real-world streams, to thoroughly examine the algorithm’s performance and adaptability. The following key conclusions can be drawn from our research:

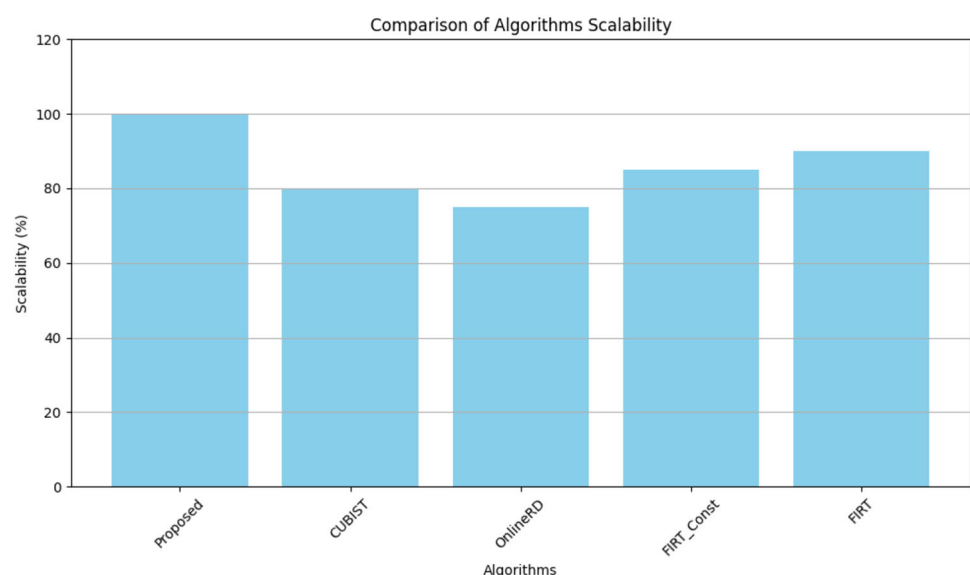
Algorithmic excellence: The Proposed Algorithm consistently outperformed ensemble models with varying ensemble sizes (5 and 10) and the EnHAT Algorithm in terms of accuracy across multiple datasets. This underscores its effectiveness and reliability in stream data classification.

Dynamic adaptation: In the dynamic context of stream data, the Proposed Algorithm demonstrated remarkable adaptability. It maintained its superior accuracy even in the presence of evolving data patterns and concept drifts. This adaptability is vital for real-world applications where timely decision-making is critical.

Real-world applicability: Our study indicates that the Proposed Algorithm holds significant promise for practical deployment. Its consistent high accuracy makes it a valuable tool in domains such as finance, healthcare, and more, where accurate and timely insights are essential.

Potential for decision support: The adaptability and robustness of the Proposed Algorithm position it as a valuable asset for decision support systems. Its ability to

Fig. 14 Compression of algorithms scalability



handle changing data patterns effectively can enhance the quality of predictions and insights.

In summary, this research contributes a robust and adaptable algorithm to the field of stream data classification. The Proposed Algorithm's consistent performance and real-world applicability make it a promising tool for dynamic data analysis and decision support systems.

5.2 Future work

While our research has provided valuable insights, several avenues for future work and research directions emerge:

Concept drift handling: Enhancing the Proposed Algorithm's concept drift detection and adaptation capabilities could further improve its performance. Investigating novel techniques for concept drift detection and adaptation is an area ripe for exploration.

Ensemble strategies: Exploring alternative ensemble strategies and their impact on accuracy and adaptability could provide valuable insights. Techniques like ensemble pruning and dynamic ensemble selection deserve attention.

Scalability: Evaluating the scalability of the Proposed Algorithm to handle even larger and more complex stream datasets is crucial. Investigating distributed and parallel computing approaches can address scalability challenges.

Real-time applications: Assessing the algorithm's suitability for real-time applications and implementing it in real-world systems is a practical next step. Collaborations with industry partners can facilitate this transition.

Interpretability: Enhancing the interpretability of the algorithm's decision-making process can foster trust and usability. Techniques for explaining model predictions should be integrated.

Domain-specific adaptations: Tailoring the algorithm to specific domains such as healthcare, finance, or cybersecurity is an exciting avenue. Domain-specific adaptations can optimize its performance for particular applications.

Human-in-the-loop integration: Investigating how human expertise can complement the algorithm's decision-making process in a human-in-the-loop framework is a promising direction. Human feedback can enhance the algorithm's adaptability.

Ethical considerations: As AI algorithms are increasingly used in decision-making, addressing ethical considerations, bias mitigation, and fairness should be integrated into future research efforts.

In conclusion, the Proposed Algorithm opens up numerous opportunities for further research and development. Addressing these future research directions can advance the field of stream data classification and its practical applications, ultimately benefiting decision-makers across various domains.

Author contributions All authors contributed to the study conception and design. Material preparation, data collection, and analysis were performed by H. Tarazodar, K. Bagherifard, and S. Nejatian. The first draft of the manuscript was written by H. Tarazodar, and all authors commented on previous versions of the manuscript. All authors read and approved the final manuscript.

Funding The authors declare that no funds, grants, or other support were received during the preparation of this manuscript.

Data availability The datasets generated during and/or analyzed during the current study are not publicly available due to [REASON(S) WHY DATA ARE NOT PUBLIC] but are available from the corresponding author on reasonable request.

Declarations

Conflict of interests The authors declare they have no relevant financial interests.

Non-financial interests The authors have no relevant non-financial interests to disclose.

References

- Abbasi A et al (2021) ElStream: an ensemble learning approach for concept drift detection in dynamic social big data stream learning. *IEEE Access* 9:66408–66419. <https://doi.org/10.1109/ACCESS.2021.3056182>
- Abu-Hashem M, Gutub A (2022) Efficient computation of Hash Hirschberg protein alignment utilizing hyper threading multi-core sharing technology. *CAAI Trans Intell Technol* 7(2):278–291. <https://doi.org/10.1049/cit2.12070>
- Amin M et al (2023) Cyber security and beyond: detecting malware and concept drift in AI-based sensor data streams using statistical techniques. *Comput Electr Eng* 108:108702. <https://doi.org/10.1016/j.compeleceng.2022.108702>
- Banar F, Tabatabaei A, Saleh M (2023) Stream data classification with Hoeffding tree: an ensemble learning approach. In: 2023 9th International conference on web research (ICWR). IEEE
- Bifet A, Gavaldà R (2009) Adaptive learning from evolving data streams. 249–260. https://doi.org/10.1007/978-3-642-03915-7_22
- Breiman L, Friedman J, Olshen R, Stone C (2017) Classification and regression trees. Chapman and Hall/CRC, New York. <https://doi.org/10.1201/9781315139470>
- Bryman A (1988) Quantity and quality in social research (1st ed.). Routledge, London. <https://doi.org/10.4324/9780203410028>
- Cohen L, Avrahami G, Last M, Kandel A (2008) Info-fuzzy algorithms for mining dynamic data streams. *Appl Soft Comput* 8(4):1283–1294. <https://doi.org/10.1016/j.asoc.2007.11.006>
- Das M, Pratama M, Savitri S, Zhang J (2019) Muse-rnn: a multilayer self-evolving recurrent neural network for data stream classification. In: 2019 IEEE international conference on data mining (ICDM). IEEE. <https://doi.org/10.1109/ICDM.2019.00028>
- Demsar J (2006) Statistical comparisons of classifiers over multiple data sets. *J Mach Learn Res* 7:1–30
- Domingos P, Hulten G (2000) Mining high-speed data streams. In: Proceedings of the sixth ACM SIGKDD international conference on knowledge discovery and data mining. ACM, pp 71–80
- Fischer L, Wollstadt P (2023) Precision and recall reject curves for classification. arXiv preprint [arXiv:2308.08381](https://arxiv.org/abs/2308.08381)

- Friedman M (1939) The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *J Am Stat Assoc* 34:109
- Friedman M (1940) A comparison of alternative tests of significance for the problem of m rankings. *Ann Math Stat* 11:86–92
- Friedman JH (1991) Multivariate adaptive regression splines. *Ann Stat* 19(1):1–67
- Gama J, Castillo G (2004) Learning with local drift detection. In: International conference on advanced data mining and applications. Springer, pp 42–55. https://doi.org/10.1007/11941439_5
- Gama J, Žliobaitė I, Bifet A, Pechenizkiy M, Bouchachia H (2014) A survey on concept drift adaptation. *ACM Comput Surv (CSUR)* 46(4):1–37. <https://doi.org/10.1145/2523813>
- Goncalves PM Jr et al (2014) A comparative study on concept drift detectors. *Expert Syst Appl* 41(18):8144–8156. <https://doi.org/10.1016/j.eswa.2014.06.022>
- Gutub A, Shambour MK, Abu-Hashem MA (2023) Coronavirus impact on human feelings during 2021 Hajj season via deep learning critical Twitter analysis. *J Eng Res* 11(1):100001
- Hemalatha J, Sekar M, Kumar C, Gutub A, Sahu AK (2023) Towards improving the performance of blind image steganalyzer using third-order SPAM features and ensemble classifier. *J Inf Secur Appl* 76:103541
- Hoeffding W (1963) Probability inequalities for sums of bounded random variables. *J Am Stat Assoc* 58:13–30. <https://doi.org/10.1080/01621459.1963.10500830>
- Hulten G, Spencer L, Domingos P (2001) Mining time-changing data streams. In: Proceedings of the seventh acm sigkdd international conference on knowledge discovery and data mining. <https://doi.org/10.1145/502512.502529>
- Ikonomovska E, Gama J, Sebastião R, Gjorgjevik D (2008) Learning model trees from data streams. In: Proceedings of the 11th international conference on discovery science. Springer, pp 52–63
- Ikonomovska E, Gama J, Sebastiao R, Gjorgjevik D (2009) Regression trees from data streams with drift detection. In: Proceedings of the international conference on discovery science. Springer, pp 121–135
- Karalic A (1992) Employing linear regression in regression tree leaves. In: Proceedings of the 10th European conference on artificial intelligence. Wiley, New York, pp 440–441
- Kelly MG, Hand DJ, Adams NM (1999) The impact of changing populations on classifier performance. In: Proceedings of the fifth ACM SIGKDD international conference on knowledge discovery and data mining. Citeseer, pp 367–371
- Kim S, Guy S, Hillesland K, Zafar B, Gutub AAA, Manocha D (2014) Velocity-based modeling of physical interactions in dense crowds. *Vis Comput* 31:541–555. <https://doi.org/10.1007/s00371-014-0946-1>
- Klinkenberg R (2004) Learning drifting concepts: example selection vs. example weighting. *Intell Data Anal* 8(3):281–300. <https://doi.org/10.3233/IDA-2004-8303>
- Kumar A, Kaur P, Sharma P (2015) A survey on Hoeffding tree stream data classification algorithms. *CPUH-Res J* 1(2):28–32
- Littlestone N, Warmuth MK (1989) The weighted majority algorithm. In: 30th annual symposium on foundations of computer science, pp 256–261
- Lucas JM, Saccucci MS (1990) Exponentially weighted moving average control schemes: properties and enhancements. *Technometrics* 32(1):1–12
- Mimran O (2013) Data stream mining with multiple sliding windows for continuous prediction. Ben-Gurion University of the Negev, Israel
- Ouyang Z, Zhou M, Wang T, Wu Q (2009) Mining concept-drifting and noisy data streams using ensemble classifiers. In: 2009 International conference on artificial intelligence and computational intelligence, vol 4. IEEE
- Pasquabisceglie V et al (2023) DARWIN: an online deep learning approach to handle concept drifts in predictive process monitoring. *Eng Appl Artif Intell* 123:106461. <https://doi.org/10.1016/j.engappai.2021.106461>
- Pfahring B, Holmes G, Kirkby R (2008) Handling numeric attributes in Hoeffding trees. In: Proceedings of the 12th Pacific-Asian conference on knowledge discovery and data mining. Springer, pp 296–307
- Potts D, Sammut C (2005) Incremental learning of linear model trees. *Mach Learn* 61:5–48. <https://doi.org/10.1007/s10994-005-1121-8>
- Pratama M, Anavatti SG, Er M, Lughofer E (2014) pClass: an effective classifier for streaming examples. *IEEE Trans Fuzzy Syst*. <https://doi.org/10.1109/TFUZZ.2014.2312983>
- Pratama M, Pedrycz W, Webb GI (2019a) An incremental construction of deep neuro-fuzzy system for continual learning of nonstationary data streams. *IEEE Trans Fuzzy Syst* 28(7):1315–1328. <https://doi.org/10.1109/TFUZZ.2019.2903985>
- Pratama M, Za'in C, Ashfahani A, Ong YS, Ding W (2019b) Automatic construction of multi-layer perceptron network from streaming examples. In: Proceedings of the 28th ACM international conference on information and knowledge management, pp 1171–1180
- Quinlan JR (1992) Learning with Continuous Classes. In: Proceedings of australian joint conference on artificial intelligence, Hobart 16–18 november 1992, pp 343–348
- Salzberg SL (1993) C4.5: Programs for machine learning by J. Ross Quinlan. Morgan Kaufmann Publishers Inc, California
- Suárez-Cetrulo A, Quintana D, Cervantes A (2022) A survey on machine learning for recurring concept drifting data streams. *Expert Syst Appl* 213:118934. <https://doi.org/10.1016/j.eswa.2022.118934>
- Svoboda R et al (2023) A natural gas consumption forecasting system for continual learning scenarios based on Hoeffding trees with change point detection mechanism. *arXiv preprint*. [arXiv:2309.03720](https://arxiv.org/abs/2309.03720)
- Tsymbal A (2004) The problem of concept drift: definitions and related work. *Comput Sci Dep Trinity Coll Dublin* 106(2):58
- Weinberg AI, Last M (2023) EnHAT—synergy of a tree-based ensemble with Hoeffding adaptive tree for dynamic data streams mining. *Inf Fus* 89:397–404. <https://doi.org/10.1016/j.inffus.2023.04.019>
- Wickham H (2011) ASA 2009 data expo. *J Comput Graph Stat* 20:281–283. <https://doi.org/10.2307/23110483>
- Wu Y et al (2023) AEWA: an efficient ensemble framework for concept drift adaptation in IoT data stream. *arXiv preprint* [arXiv:2305.06638](https://arxiv.org/abs/2305.06638)
- Xu Y, Xu R, Yan W, Ardis P (2017) Concept drift learning with alternating learners. In: 2017 International joint conference on neural networks (IJCNN). IEEE, pp 2104–2111
- Zenisek J, Holzinger F, Affenzeller M (2019) Machine learning based concept drift detection for predictive maintenance. *Comput Ind Eng* 137:106031. <https://doi.org/10.1016/j.cie.2019.07.005>
- Zliobaite I (2010) Learning under concept drift: an overview. *arXiv preprint* [arXiv:1010.4784](https://arxiv.org/abs/1010.4784)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.